

向量

ADT：接口与实现

02-A1

邓俊辉

deng@tsinghua.edu.cn

劳动生产力上最大的增进，以及运用劳动时所表现的更大的
熟练、技巧和判断力，似乎都是分工的结果

Abstract Data Type vs. Data Structure

抽象数据类型 = 数据模型 + 定义在该模型上的一组操作

抽象定义

一种定义

外部的逻辑特性

不考虑时间复杂度

操作&语义

不涉及数据的存储方式

数据结构 = 基于某种特定语言, 实现ADT的一整套算法

具体实现

多种实现

内部的表示与实现

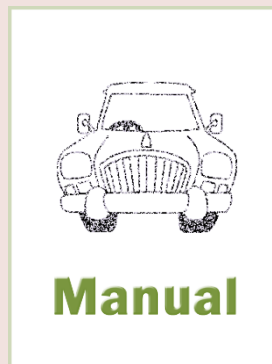
与复杂度密切相关

完整的算法

要考虑数据的具体存储机制



Application



ADT/Interface



Implementation

Application = Interface x Implementation

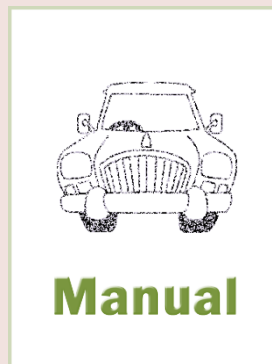
❖ 在数据结构的具体实现与实际应用之间

ADT就分工与接口制定了统一的规范

- 实现：高效兑现数据结构的ADT接口操作
//做冰箱、造汽车
- 应用：便捷地通过操作接口使用数据结构
//用冰箱、开汽车



Application



ADT/Interface



Implementation

❖ 按照ADT规范

- 高层算法设计者可与
底层数据结构实现者高效地分工协作
- 不同的算法与数据结构可以便捷组合借用
- 每种操作接口只需统一地实现一次
代码篇幅缩短，安全性加强，软件复用度提高

向量

ADT: 从数组到向量

02-A2

秩秩斯干，幽幽南山

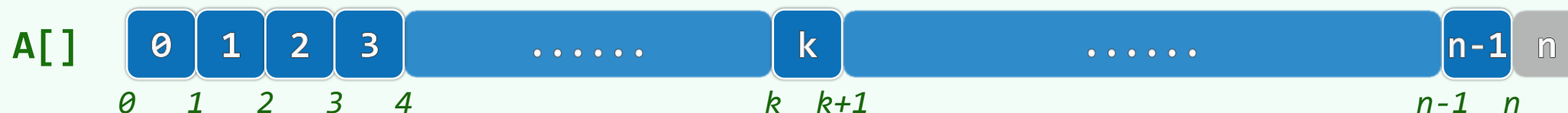
到了，就是这里。妈妈，你只管找号头，311，就是爸爸的号

邓俊辉

deng@tsinghua.edu.cn

数组 ~ 循序访问 ~ 向量

❖ C/C++语言中，数组元素与编号一一对应：A[0], A[1], A[2],, A[n-1]



若每个元素占用的空间量为s，则A[i]的**物理地址** = A + i×s //可能有padding

❖ 向量是数组的**抽象与泛化**，由一组元素按线性次序**封装**而成

各元素与[0,n)内的**秩** (rank) 一一对应： using Rank = uint32_t; //call-by-rank

❖ 操作、管理维护更加简化、统一与安全；元素类型可灵活**选取**，便于**定制**复杂数据结构：

Vector<BTNodePosi<T>> child; //B-树中，节点用向量记录所有的孩子

Vector<Vector<Edge<Te>*>> E; //图结构中，用二维向量实现邻接矩阵，存放图中的边

应用者视角：Vector ADT

操作接口	功能
<code>size()/empty()</code>	报告元素总数/判断向量是否为空
<code>insert(r,e)</code>	将e作为秩为r的元素插入（后继元素顺次后移）
<code>remove(r)</code>	删除秩为r的元素（后继元素顺次前移）
<code>sort()/unsort()</code>	排序/置乱
<code>find(e)/search(e)</code>	在无序/有序向量中查找e
<code>dedup()/uniquify()</code>	从无序/有序向量中剔除相等的元素
<code>traverse(visit())</code>	对每个元素统一执行visit操作

应用者视角：ADT操作实例

操作	输出	向量内容
初始化构造		空
size()	0	^
empty()	true	^
insert(0, 'N')		N
insert('A')		N A
insert(0, 'D')		D N A
size()	3	^
insert(1, 'A')		D A N A
remove(2)	'N'	D A A
insert(2, 'T')		D A T A
size()	4	^

操作	输出	向量内容
find('N')	-1	D A T A
find('T')	2	^
find('A')	3	^
dedup()	1	D A T
insert(3, 'A')		D A T A
sort()		A A D T
search('A')	1	^
search('D')	2	^
search('S')	2	^
search('X')	3	^
uniquify()	1	A D T

STL Vector

```
#include <iostream>
```

```
#include <vector>
```

```
using namespace std;
```

```
vector<int> v; //an empty vector of integers
```

```
vector<int> s( 81, 25 ); //{ 25, 25, 25, 25, 25, 25, ..., 25 }
```

```
s.insert( s.begin() + 5, 2025 ); //{ 25, 25, 25, 25, 25, 2025, 25, ..., 25 }
```

```
s.erase( s.end() - 75, s.end() ); //{ 25, 25, 25, 25, 25, 2025, 25 }
```

```
for ( int i = 0; i < s.size(); i++ )
```

```
    cout << s[i] << endl;
```


向量

ADT: 模板类

02-A3

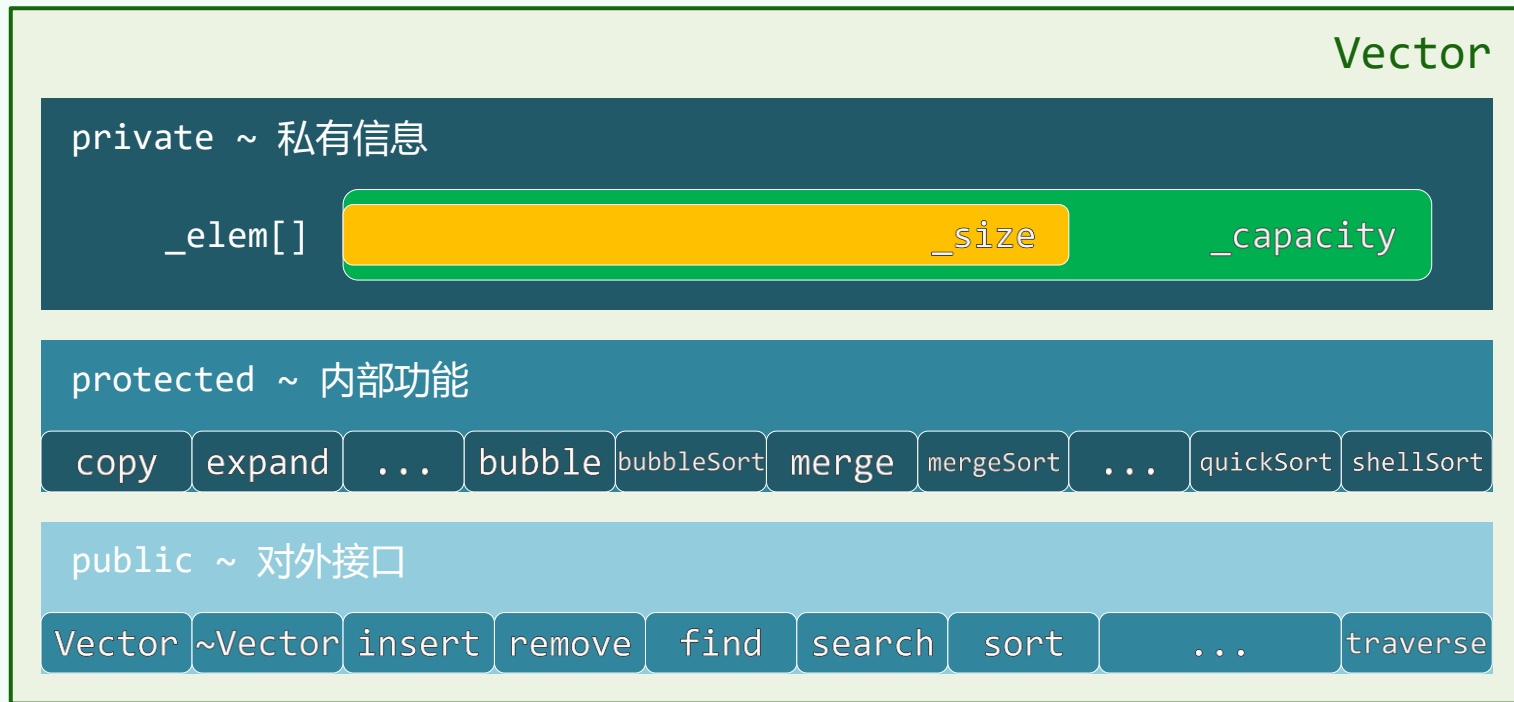
邓俊辉

deng@tsinghua.edu.cn

官職須由生處有，文章不管用時無
堪笑翰林陶學士，年年依樣畫葫蘆

实现者视角：Vector模板类

```
template <typename T>
class Vector { //向量模板类
private:
    T* _elem; //数据区
    Rank _capacity; //容量
    Rank _size; //规模
protected:
    /* ... 内部接口 */
public:
    /* ... 开放接口 */
};
```



循秩访问

❖ 可否沿用**数组**的方式，便捷、直观地通过**V[r]**来访问向量**V**中的第**r**个元素？

可以！比如，通过**重载**下标操作符**[]**

❖ `template <typename T> //可作为左值: V[r] = (T) (2*x + 3)`

```
T & Vector<T>::operator[] ( Rank r ) { return _elem[ r ]; }
```

❖ `template <typename T> //仅限于右值: T x = V[r] + U[s] * W[t]`

```
const T & Vector<T>::operator[] ( Rank r ) const { return _elem[ r ]; }
```

❖ 我们将采用**简易**的方式处理**意外和错误**（比如，入口参数约定： $0 \leq r < _size$ ）

构造 + 析构：重载

```
#define DEFAULT_CAPACITY 2 //默认初始容量（小数值利于充分测试；实际应用中可设置为更大）
```

```
Vector( Rank c = DEFAULT_CAPACITY ) //开辟空间，并按默认方法构造每一个成员对象
```

```
{ _elem = new T[ _capacity = c ]; _size = 0; }
```

```
Vector( Rank c, Rank s, T v ) : Vector( c ) //s <= c: 创建一个容量为c、规模为s
```

```
{ for ( _size = 0; _size < s; _elem[ _size++ ] = v ); } //所有元素均为v的向量
```

```
Vector( const T * A, Rank lo, Rank hi ) { copy( A, lo, hi ); } //通过复制来创建
```

```
Vector( Vector<T> const & V, Rank lo, Rank hi ) { copy( V._elem, lo, hi ); }
```

```
Vector( T const* A, Rank lo, Rank hi ): Vector( (hi-lo)*2 ) { copy( A, lo, hi ); }
```

```
~Vector() { delete [] _elem; } //析构所有成员对象，并释放空间
```

基于复制的构造

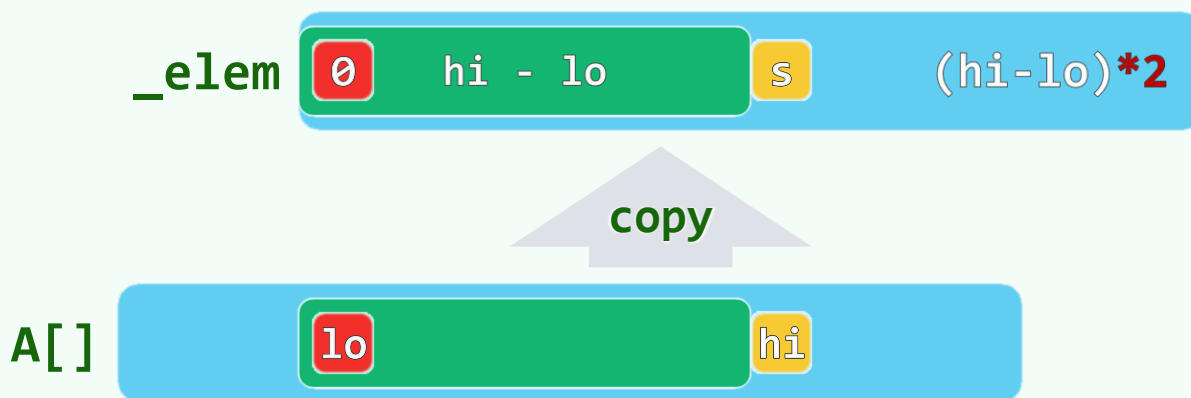
```
template <typename T>
```

```
void Vector<T>::copy( const T * A, Rank lo, Rank hi ) {
```

```
    for ( _size = 0; lo < hi; _size++, lo++ ) //A[lo,hi)复制至_elem[0,hi-lo)
```

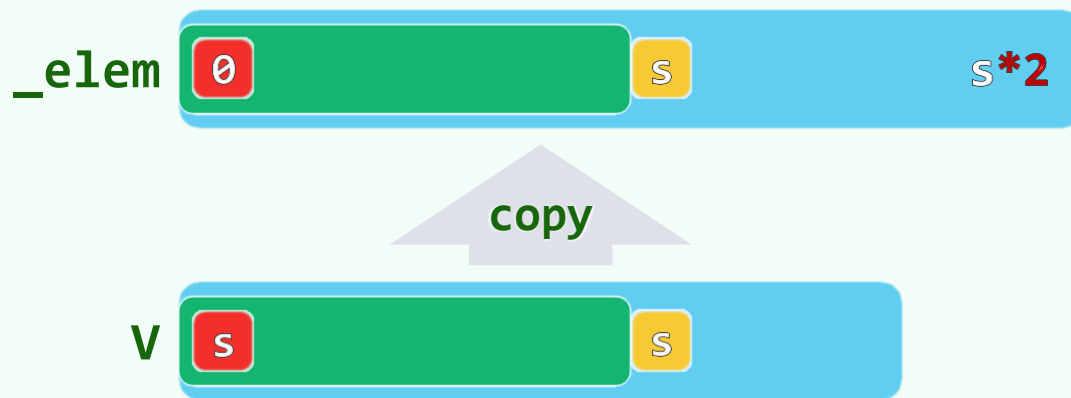
```
        _elem[ _size ] = A[ lo ]; //T若非基本类型, 则需重载操作符 '=' 以实现深复制
```

```
} //copy, O(hi-lo)
```



深复制

```
template <typename T> Vector<T>& Vector<T>::operator=( Vector<T> const& V ) {  
    _size = 0; //删除原有内容  
  
    expand( V._size ); copy( V._elem, 0, V._size ); //整体复制  
  
    return *this; //返回当前向量的引用，以便链式赋值  
} //operator=
```



向量

无序向量：插入与删除

02-B1

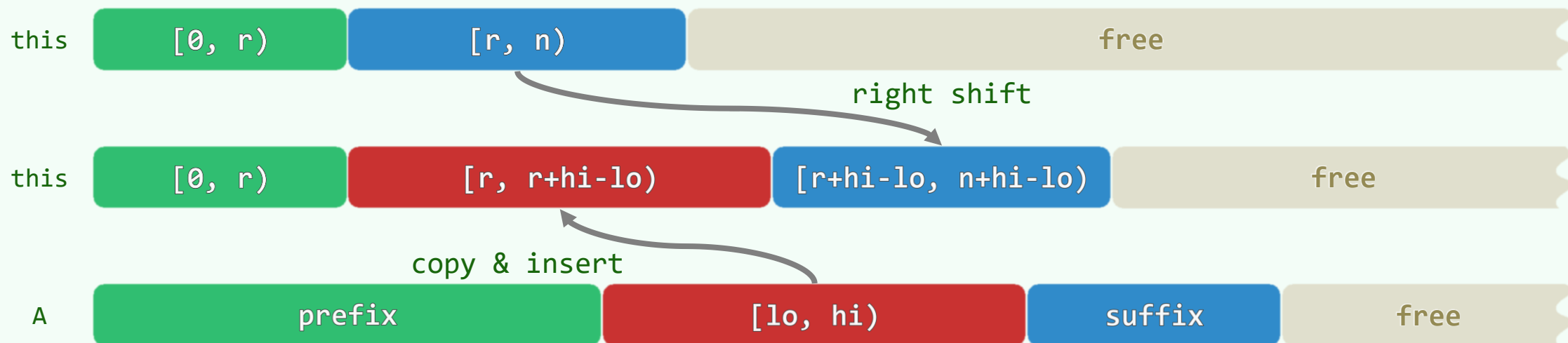
A scientist discovers that which exists,
an engineer creates that which never was.

邓俊辉

deng@tsinghua.edu.cn

插入：自后向前地后移

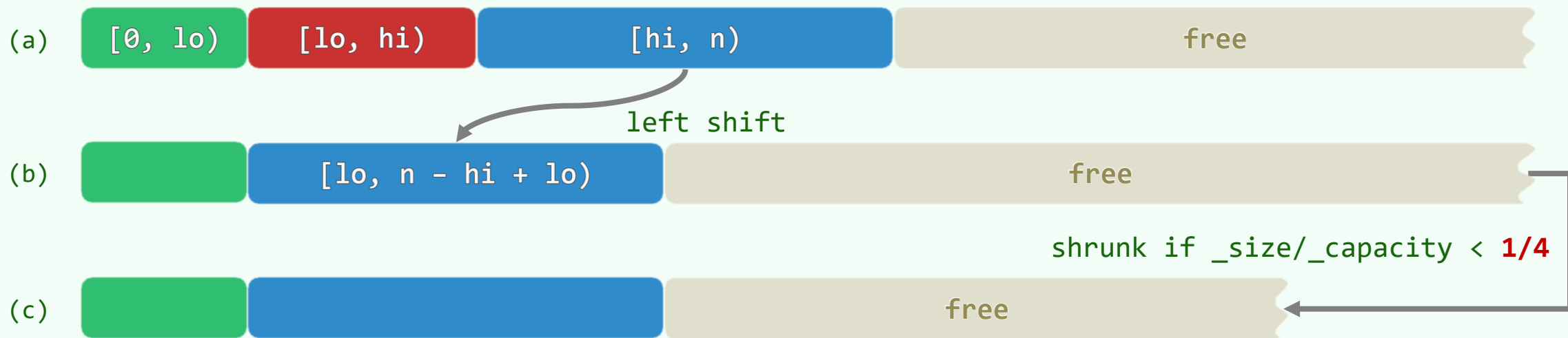
```
template <typename T> void Vector<T>::insert( Rank r, const T * A, Rank lo, Rank hi ) {  
    Rank d = hi - lo;    expand( d ); //若必要，先扩容：确保足以新增d个元素  
    for ( Rank i = _size-1; i != r-1; i-- ) _elem[i+d] = _elem[i]; //长度为n-r的后缀：后移  
    while ( lo < hi ) _elem[r++] = A[lo++];    _size += d; //插入d个元素  
}
```



```
template <typename T> Rank Vector<T>::insert( Rank r, const T & e ) { insert(r, &e, 0, 1); }
```


删除：自前向后地前移

```
template <typename T> void Vector<T>::remove( Rank lo, Rank hi ) { //0<=lo<hi<=n
    while ( hi < _size ) _elem[ lo++ ] = _elem[ hi++ ]; //O(n-hi): 正比于后缀长度, 而与hi-lo无关
    _size = lo;    shrink(); //更新规模, 可能缩容
}
```



```
template <typename T> T Vector<T>::remove( Rank r )
{ T e = _elem[r];    remove( r, r+1 );    return e; } //O(n-r), 仍正比于后缀长度
```

性能分析

❖ **单元素**的插入|删除，为何都实现为**区间**插入|删除的特例？ //General --> Specific

反过来，为何不将后者视作前者的**多次**重复呢？ //Specific --> General

❖ 以remove()为例...

❖ G2S：后缀一次性地大跨步前移

时间成本仅决定于**后缀长度**，而与删除的**元素数量**无关： $_{size} - hi = \mathcal{O}(n)$

❖ S2G：后缀**反复地小步**前移： $(_{size} - hi) \cdot (hi - lo) = \mathcal{O}(n^2)$

❖ insert()总体亦类似（尽管略有区别）

❖ 经验：重复性的任务应尽可能**集中**起来，**批量**地完成 //作业例外

向量

无序向量：扩容

02-B2

一個人辦一縣事，要有一省的眼光；辦一省事，要有一國之眼光；辦一國事，要有世界的眼光

其实“我”不需扩大，宇宙只是一个“我”，只有在我们精神往下陷落时，宇宙与我才分开

邓俊辉

deng@tsinghua.edu.cn

空间管理

❖ 申请空间时留有足够**余地**，以保证在相当长的时间内无需**追加**空间



❖ 然而，对于任何固定的容量 `_capacity`，总是难免会出现尴尬情况...

❖ 定义：**装填因子**/load factor = $\lambda = \text{_size} / \text{_capacity}$

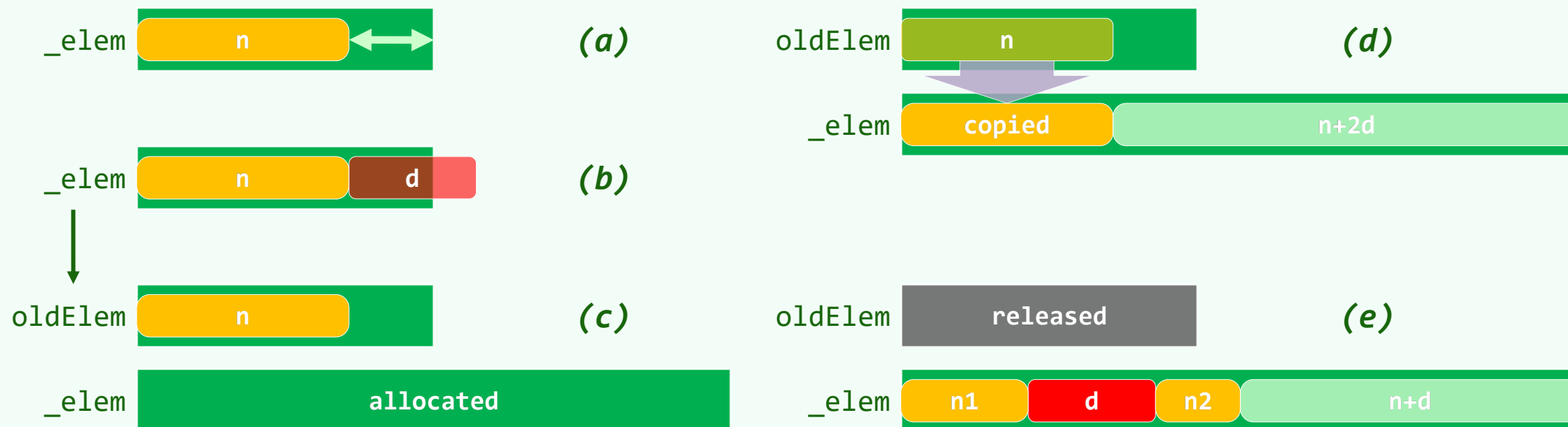
- 上溢/overflow: 不足以接纳新的元素: $\lambda = 100\%$
- 下溢/underflow: 实际存放的元素寥寥无几: $\lambda \ll 50\%$

❖ 为保证向量持续高效运转，需令其根据实际情况**动态调整**容量，始终保证

25% $\leq \lambda \leq$ **100%** ...

动态扩容

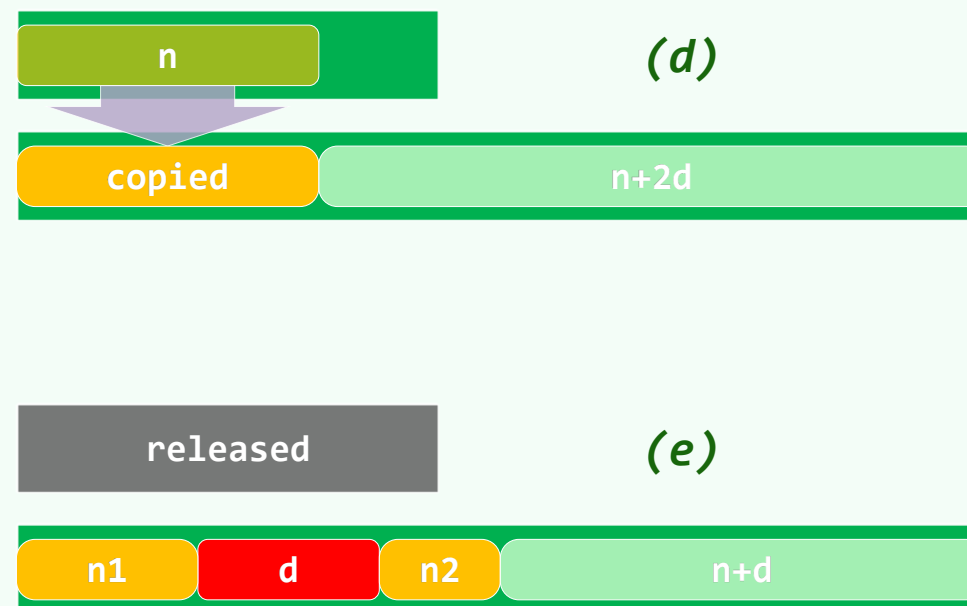
❖ 蝉：身体经过一段时间的生长，会蜕去原先的外壳，代之以**更大**的新外壳



❖ 向量：在即将**上溢**时，**适当扩大**内部数据区的容量

扩容算法

```
template <typename T> void Vector<T>::expand( Rank d = 1) {  
    if ( _size + d <= _capacity ) return; //若足以新增d个元素，则无需扩容  
  
    T* oldElem = _elem; //记下原数据区  
  
    _capacity = ( _size + d ) * 2; //容量加倍  
  
    _elem = new T[ _capacity ]; //分配新空间  
  
    copy( oldElem, 0, _size ); //转移数据  
  
    delete [] oldElem; //释放原空间  
  
} //为何采用了容量加倍策略?
```



❖ 最好 $O(1)$ ，最坏 $O(n)$ ，相差悬殊，且非此即彼——如何从整体上界定其性能呢？

向量

无序向量：均摊分析

02-B3

邓俊辉

deng@tsinghua.edu.cn

那时她的儿子还太年轻……一心以为自己是世上最不幸的一个，
不知道儿子的不幸在母亲那儿总是要加倍的

平均分析?

❖ **平均复杂度**|average complexity ~ **期望复杂度**|expected complexity

- 根据各种操作出现的**概率**，对其运行时间做**加权**平均

❖ 很遗憾，平均分析法并不适用于expand()，因为...

- 其最好、最坏情况的出现并非**随机**的

❖ 对任何一个**真实**的向量，考查其**完整的、足够长**的历史

(简化起见，设仅有插入而无删除，且都是单元素插入)

- 截至 `_size = n >> 2` 时，必已做过 `n` 次 `insert()`，进而 `expand()` 也被调用过 `n` 次
- 纵观这 `n` 次调用，它们其实是完全**确定**的...

均摊分析!

❖ ...

- 每出现一次 $O(n)$ 的情况, `_capacity`总会**加倍**, 随后的装填因子总是**50%**
- 于是, 接下来需要再做 n 次插入, 才会再次因上溢而出现 $O(n)$ 的情况
- 而此前对应的 n 次`expand()`调用必然都属于 $O(1)$ 的情况

❖ 这样的—个向量对`expand()`的**连续**调用, **累积**的时间成本总是决定于**初始状态**, 以及**调用次数**

❖ **均摊复杂度** | amortized complexity

- 考查对—个**真实算法****连续的**、**足够多次**的调用, 将其间**总共**花费的时间, **摊还**至单次调用

如此, 方能更加精准、更加理性地评判—个数据结构和算法的真实性能

容量加倍策略

```
_capacity = (_size + d) * 2;
```

❖ 不妨假设

- 从容量为0的空向量开始
- 逐个地插入 $n = 2^m - 1$ 个元素，而无删除操作

❖ 最好情况至多 n 次，累计耗时不超过 $\mathcal{O}(n)$

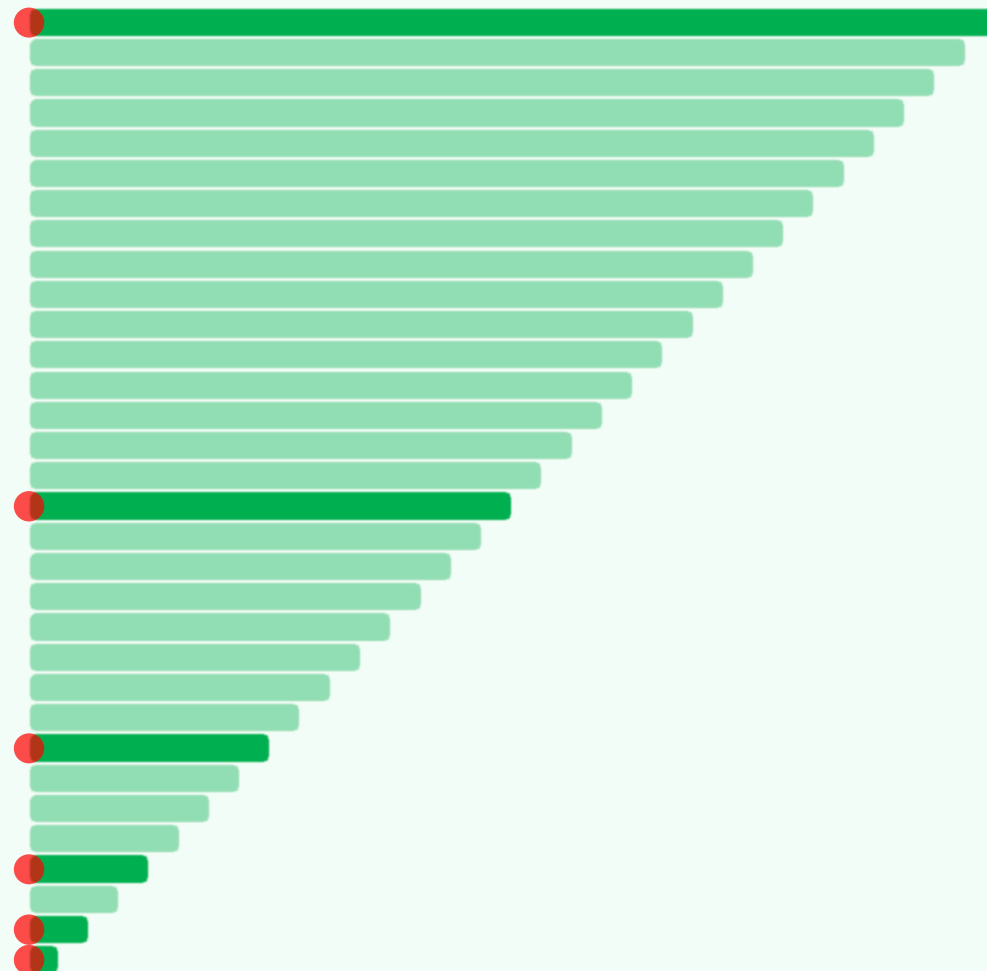
❖ 最坏情况出现在第1, 3, 7, 15, 31, ...次插入

(分别扩容至 $_capacity = 2, 6, 14, 30, 62, \dots$)

❖ 时间主要消耗于数据的复制搬迁

大致构成一个以2为倍数的几何级数，累计 $\mathcal{O}(n)$

合计之后，摊还至每一次 expand() 调用，只有 $\mathcal{O}(1)$!



容量递增策略

```
_capacity += INCREMENT;
```

❖ 考查同样的情况，增量简记作I

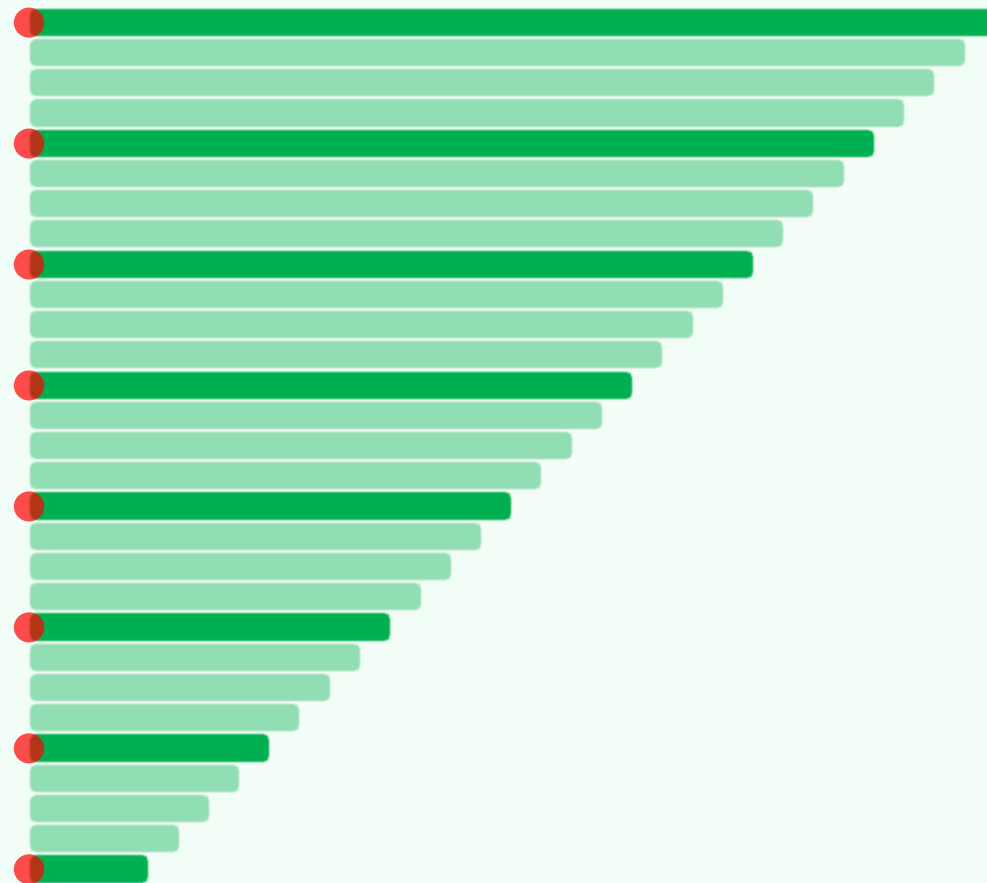
- 忽略**最好**情况
- 每隔I次调用，便会出现一次**最坏**情况

❖ 时间依然主要消耗于数据的**复制**搬迁

构成一个以I为增量的**算术级数**，累计

$$\Omega(n^2/I/2) = \Omega(n^2)$$

摊还至每一次expand()调用，高达 $\Omega(n)$!



向量

无序向量：查找与去重

君子和而不同，小人同而不和

他便站将起来，背着手踱来踱去，侧眼把那些人逐个个觑将去，
内中一个果然衣领上挂着一寸来长短彩线头

鳳兮鳳兮，故是一鳳

你去問問你璉二孀子，正月裡請吃年酒的日子擬了沒有。若擬定了，叫
書房裡明白開了單子來，咱們再請時，就不能重犯了。舊年不留心重了
幾家，不說咱們不留神，倒象兩宅商議定了送虛情怕費事一樣

邓俊辉

deng@tsinghua.edu.cn

判等器 ~ 比较器

```
template <typename K, typename V> struct Entry { //词条模板类
    K key; V value; //关键码、数值
    Entry ( K k = K(), V v = V() ) : key ( k ), value ( v ) {}; //默认构造函数
    Entry ( Entry<K, V> const& e ) : key ( e.key ), value ( e.value ) {}; //克隆

    // 被判为相等的词条，未必相同
    bool operator== ( Entry<K, V> const& e ) { return key == e.key; } //等于
    bool operator!= ( Entry<K, V> const& e ) { return key != e.key; } //不等于

    // 得益于比较器和判等器，从此往后，不必严格区分词条及其对应的关键码
    bool operator< ( Entry<K, V> const& e ) { return key < e.key; } //小于
    bool operator> ( Entry<K, V> const& e ) { return key > e.key; } //大于
}; //Entry
```

顺序查找

```
template <typename T> //0 <= lo < hi <= _size
```

```
Rank Vector<T>::find( const T & e, Rank lo, Rank hi ) const { //O(hi-lo)
```

```
    while ( (lo < hi--) && (e != _elem[hi]) ); //从后向前，逐个比对
```

```
    return hi; //最靠后的命中者，或lo-1示意失败 (lo==0时为-1 == UINT_MAX)
```

```
} //find
```



❖ 输入敏感|input-sensitive: 最好 $O(1)$, 最差 $O(n)$, 有实质区别

去重：减而治之

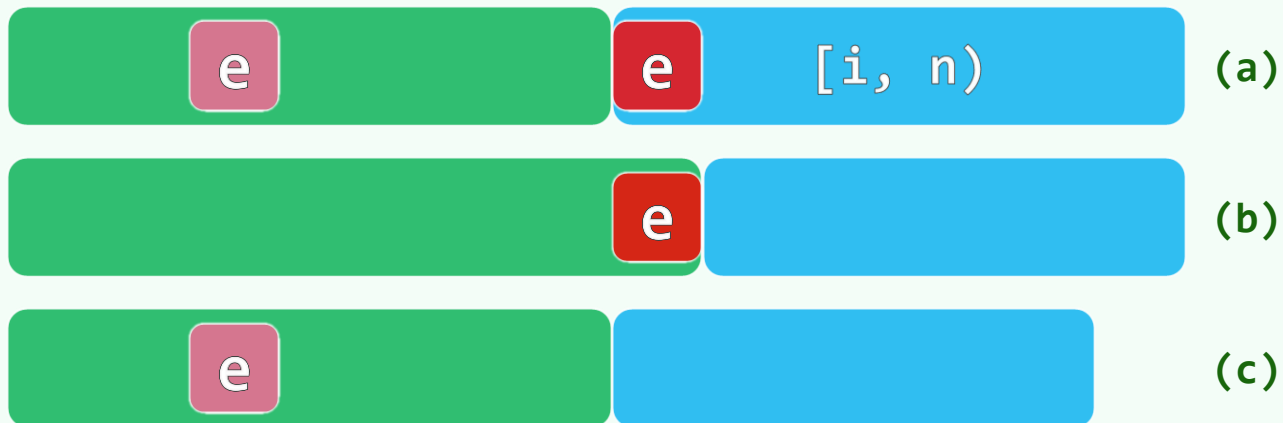
```
template <typename T> Rank Vector<T>::dedup() { //剔除相等的元素

    for ( Rank i = 1; i < _size; )

        if ( -1 == find( _elem[i], 0, i ) ) i++; //O(i)

        else remove(i); //O(n-i)

} //O(n^2)
```



向量

无序向量：遍历

02-B5

邓俊辉

deng@tsinghua.edu.cn

让他们每个人轮流到你的宝座下，同样诚恳地坦白他们的内心，
然后再看有没有一个人敢向你说：“我比这个人好。”

遍历：对向量中的每一元素，实施统一的操作

❖ 如果遍历是列队操演

那么统一的操作就是号令：

- 立正、稍息、向右看齐、报数...

❖ 号令如何描述？如何传达至每个元素？

❖ 函数对象 | Function Object

- 本是一个对象 `//visit`

- 但在重载了 “()” 之后

等效于一个函数 `//visit()`

```
template <typename T> template <typename VST>
void Vector<T>::traverse( VST& visit ) {
    for ( Rank i = 0; i < _size; i++ )
        visit( _elem[i] );
}
```

单兵动作：统一地将向量中的元素各自加一

❖ 先实现一个类，重载圆括号，可使单个T类型的元素加一

```
template <typename T> //假设T可直接递增或已重载操作符 “++”  
  
struct Increase //函数对象：通过重载操作符 “()” 实现  
  
    { virtual void operator()( T & e ) { e++; } }; //加一
```

❖ 再将其作为参数，传递给遍历算法

```
template <typename T> void increase( Vector<T> & V )  
  
    { V.traverse( Increase<T>() ); } //即可以之作为基本操作，遍历向量
```

❖ 课后练习：模仿此例，实现统一的减一、加倍等更多的遍历功能

整体动作：统计元素总和，作为校验值

```
❖ template <typename T> struct Cumulate { //累计一个T类型对象的数值
    T & s;

    Cumulate( T & sum ) : s( sum ) {}

    virtual void operator()( T & e ) { s += e; } //约定T可直接相加
}; //Cumulate

❖ template <typename T> T sum( Vector<T> & V ) { //统计向量中所有元素的总和
    T sum = 0;

    V.traverse( Cumulate<T>( sum ) ); //以Cumulate为基本操作进行遍历

    return sum; //返回总和
} //sum
```

复杂整体动作：统计逆序对总数，判断是否有序

```
❖ template <typename T> struct Compare { //判断线性序列中的一个元素是否与其前驱顺序
    T pred; int & i; //前驱、（相邻）逆序对的计数器
    Compare( int & invs, T & first ) : pred( first ), i( invs ) {}
    virtual void operator()( T & e ) { i += ( pred > e ); pred = e; }
}; //Compare

❖ template <typename T> int inv( Vector<T> & V ) { //统计向量中相邻逆序对的总数
    int invs = 0;
    V.traverse( Compare<T>( invs, V[0] ) ); //以Compare为基本操作做遍历
    return invs; //返回逆序对总数：为零当且仅当向量整体有序
} //inv
```

向量

有序向量：去重

02-C1

面壁计划已经恢复，您被指定为唯一的面壁者

我要你另換個主意，不許雷同了前人，只做個破題也使得

習藝愈勤，去修養愈遠。何以故？曰，無閒暇故

邓俊辉

deng@tsinghua.edu.cn

勤奋的低效算法

❖ 有序向量中的每一组**重复**元素，都会彼此聚集地占据一个**区段**

去重的效果可以理解为，从每个区段中**删除**第一个之后的所有元素

❖ `template <typename T> void Vector<T>::uniquify() {`

`for ( Rank i = 1; i < _size; ) //从前向后，逐一比对相邻的[i-1]和[i]`

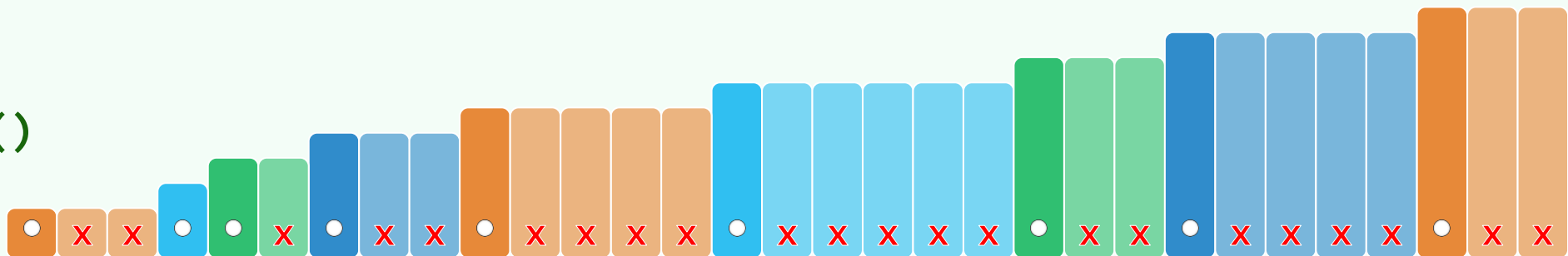
`if ( _elem[i-1] != _elem[i] ) i++; //若不等，则转至下一对元素`

`else remove( i ) : //否则，删除居后者`

`} //O(n^2)`

❖ 耗时的remove()

调用太多!



懒惰的高效算法: Two-Pointer Technique

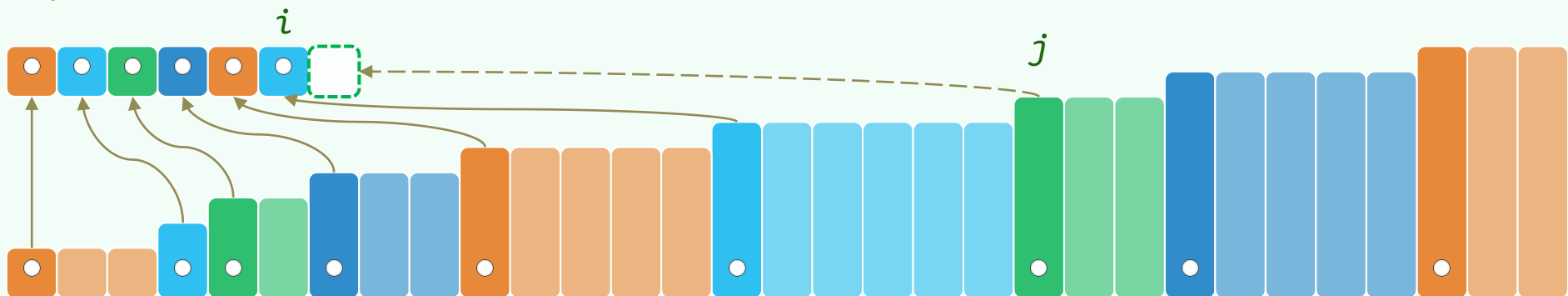
- ❖ 调转视角，去重的效果也可等效地理解为，仅保留各区段的首元素

- ```
❖ template <typename T> void Vector<T>::uniquify() {
 Rank i = 0; //前缀[0,i]包含目前已知的所有无重元素
 for (Rank j = 1; j < _size; j++) //逐一考查[j]
 if (_elem[i] != _elem[j]) //不等, 当且仅当[j]为一个区间的首元素
 _elem[++i] = _elem[j]; //故将[j]归入无重的前缀
```

```
_size = ++i;
```

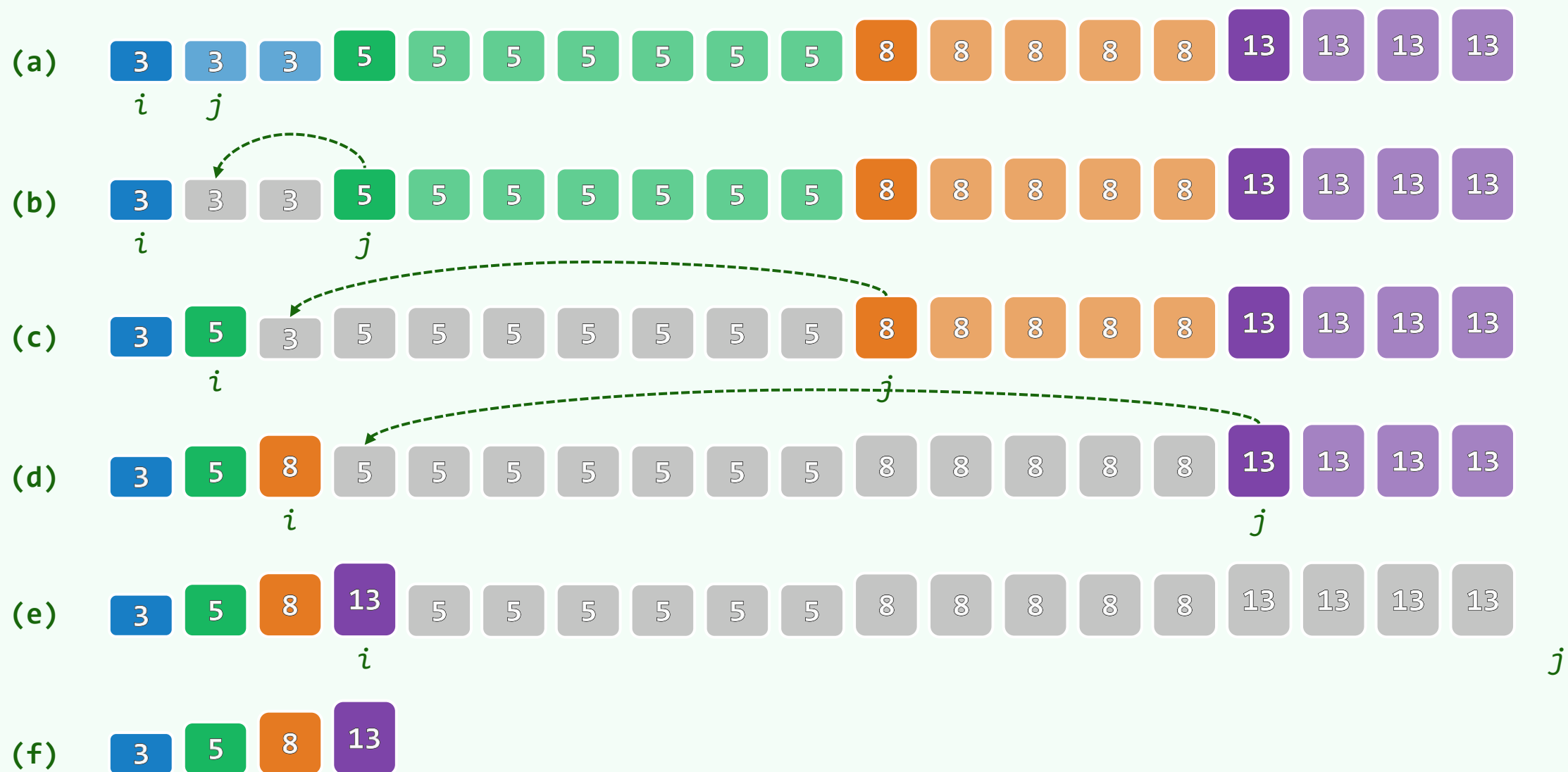
shrink();  <sup>i</sup> 

```
} // O(n)
```



- ❖ 每一步仅 $O(1)$

# 实例





向量

有序向量：二分查找（版本A）

02-02

自從爺爺去後，這山被二郎菩薩點上火，燒殺了大半...及至火滅煙消，出來時，又沒花果養贍，難以存活，別處又去了一半。我們這一半，捱苦的住在山中，這兩年，又被些打獵的搶了一半去也

邓俊辉

deng@tsinghua.edu.cn

# 接口与算法

❖ `template <typename T> //整体查找`

```
Rank search(const T& e) const { return search(e, 0, _size); }
```

`[0,lo)`

`[lo,hi)`

`[hi,n)`

❖ `template <typename T> //区间查找:  $0 \leq lo < hi \leq \_size$`

```
Rank Vector<T>::search(const T & e, Rank lo, Rank hi) const {
```

```
 return (rand() % 2) ? //为便于测试, 暂且随机地调用
```

```
 binSearch(_elem, e, lo, hi) : //二分查找算法, 或
```

```
 fibSearch(_elem, e, lo, hi) ; //Fibonacci查找算法
```

```
} //在实际应用中, 你可针对具体问题选用最适宜的一种
```

## 轴点 ~ 减治

❖ 有序向量中，每个元素都是**轴点**： $[lo, mi) \leq [mi] \leq (mi, hi)$ ，因此...

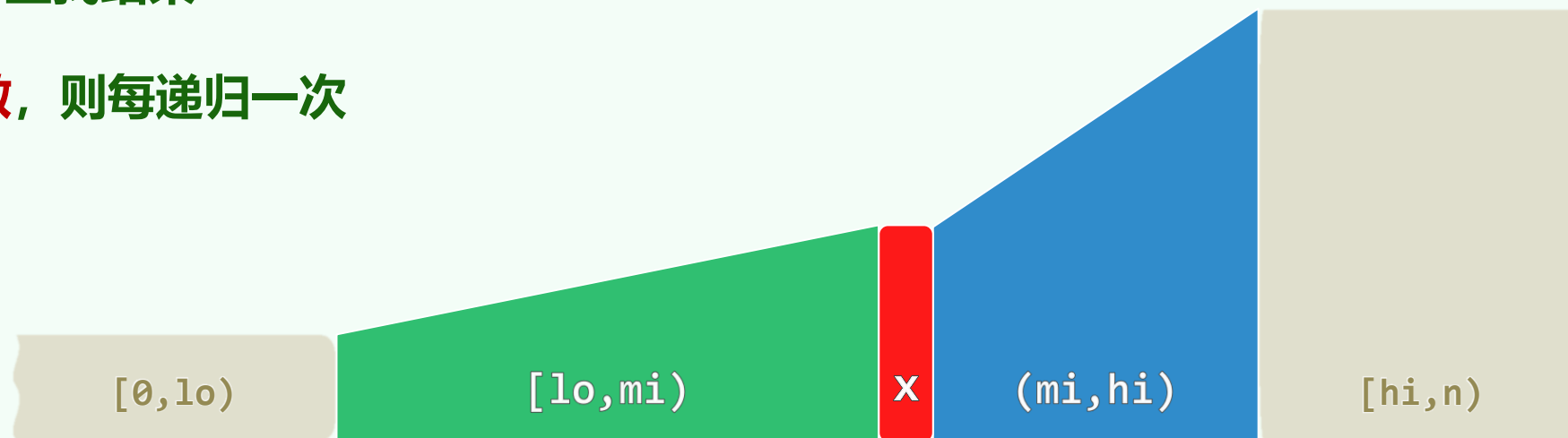
❖ 视目标元素 $e$ 与 $[mi]$ 的相对大小，无非**三种**情况：

- $e < [mi]$ ： $e$ 只能存在于**前缀**中，故可减除 $[mi, hi)$ 并递归深入 $[lo, mi)$
- $[mi] < e$ ： $e$ 只能存在于**后缀**中，亦可减除 $[lo, mi]$ 并递归深入 $(mi, hi)$
- $e = [mi]$ ：**命中**，查找结束

❖ 若 $[mi]$ 总是取作**中位数**，则每递归一次

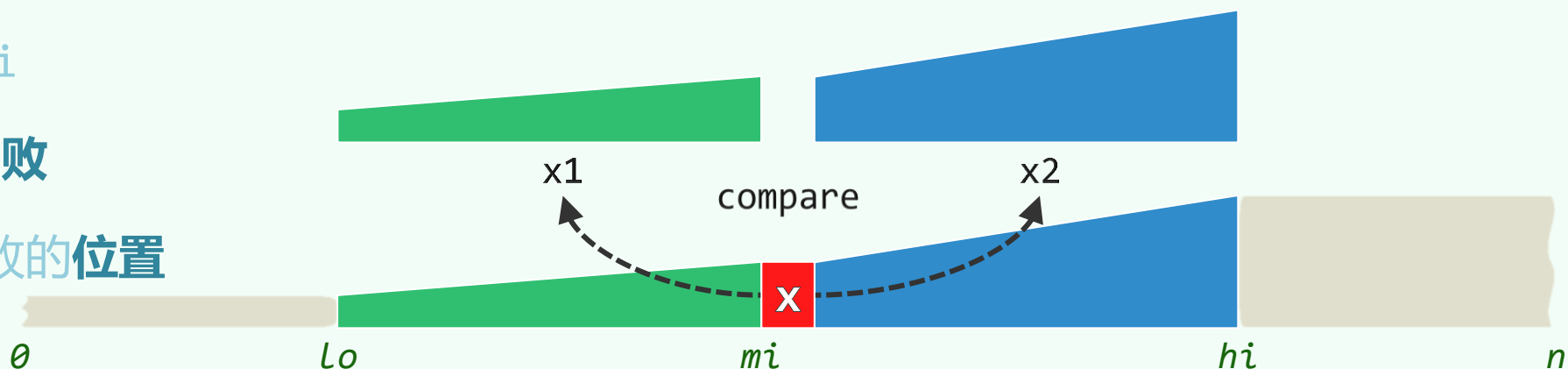
- 要么问题规模**减半**
- 要么能够命中

递归深度 $O(\log n)$



# 实现

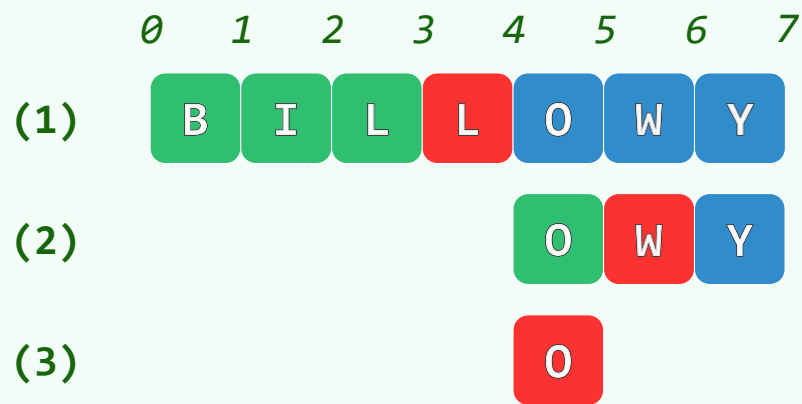
```
template <typename T> Rank binSearch(T* S, const T& e, Rank lo, Rank hi) {
 while (lo < hi) { //每步迭代三个分支, 最多两次比较
 Rank mi = (lo + hi) / 2; //以中位数为轴点, 区间宽度折半
 if (e < S[mi]) hi = mi; //深入前缀[lo, mi)
 else if (S[mi] < e) lo = mi + 1; //深入后缀(mi, hi)
 else return mi; //命中, 收工: 有多个e时, 不能保证返回秩最大者
 } //出口时lo==hi
 return -1; //失败
} //但-1不能指示失败的位置
```



## 实例 + 复杂度

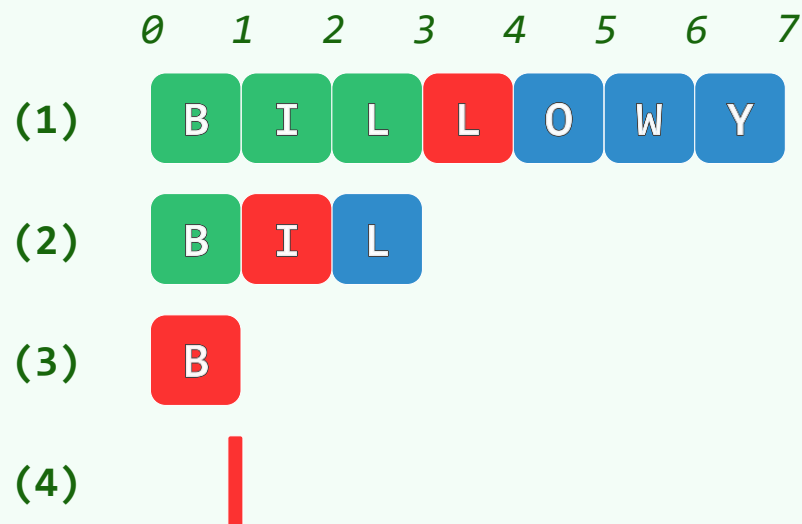
`binSearch('O')`

经 $2+1+2=5$ 次比较, 命中于[4]



`binSearch('F')`

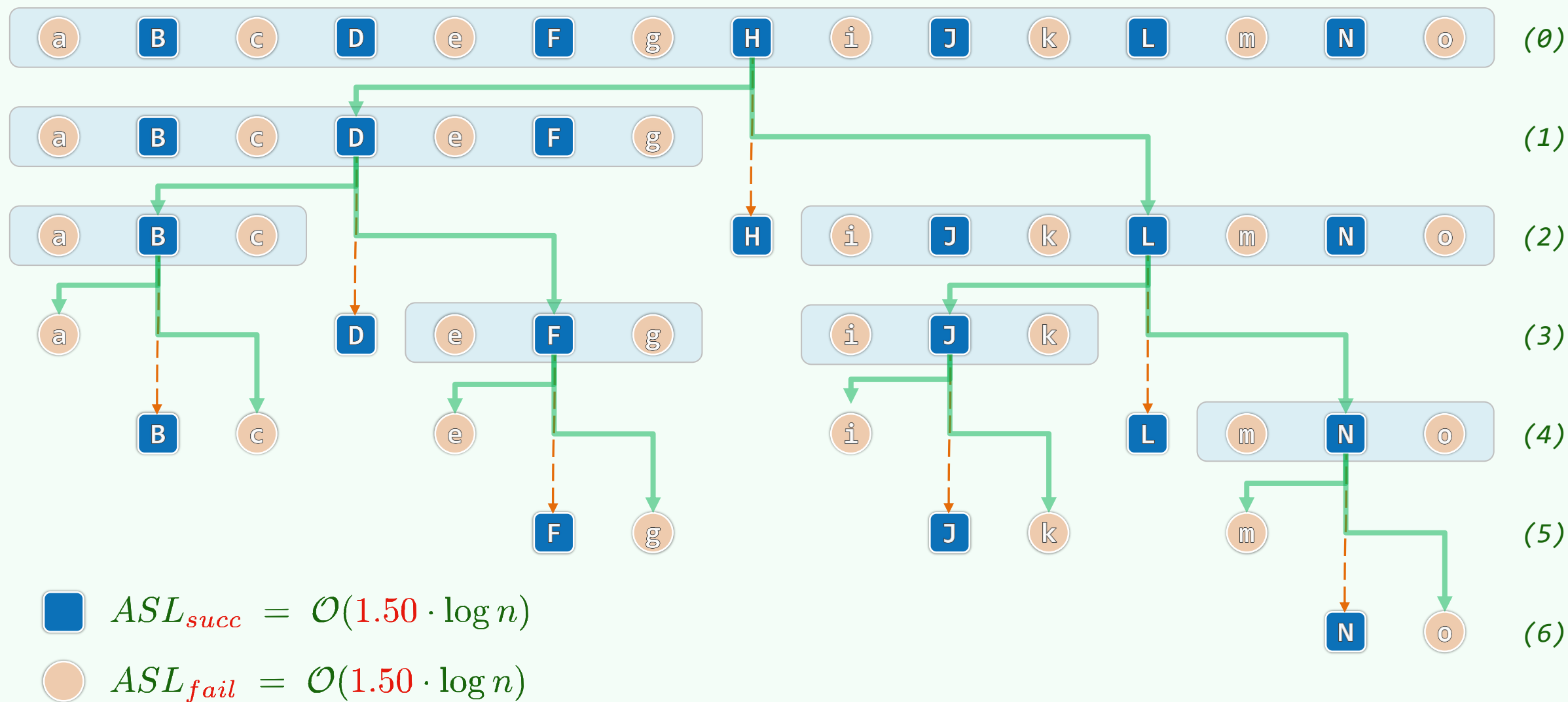
经 $1+1+2=4$ 次比较, 失败于[1]



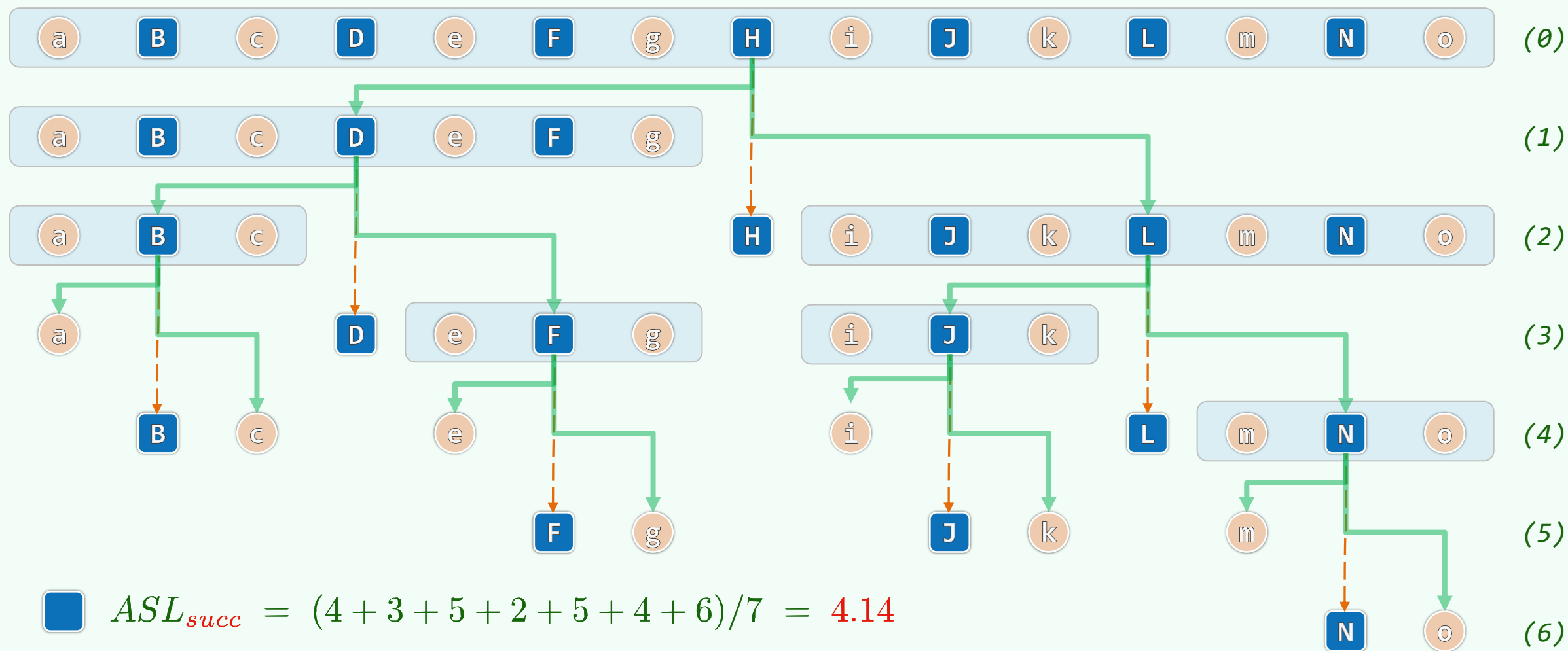
❖ 线性递归:  $T(n) = T(n/2) + \mathcal{O}(1) = \mathcal{O}(\log n)$ , 大大优于顺序查找

递归跟踪: 深度  $\mathcal{O}(\log n)$ ; 各递归实例仅耗时  $\mathcal{O}(1)$

# 元素比较的次数 ~ 平均查找长度 (Average Search Length)



# 实例：n = 7，按等概率估算



向量

有序向量：二分查找（版本B）

02-03

邓俊辉

deng@tsinghua.edu.cn

和微风匀到一起的光，象冰凉的刀刃儿似的，把宽静的大街切成两半，一半儿黑，一半儿亮。那黑的一半，使人感到阴森，亮的一半使人感到凄凉



## 改进思路

❖ 事实：查找成功的**概率**通常极低：对于 $n=10^6$ 个int组成的有序向量，不足  $10^6/2^{32} \approx 1/4000$

悖论：为了如此**难得**一现的情况，此前的算法居然在每一步迭代中都花费时间做了一次**比较**！

❖ 启示：与其贪恋虚妄、踟蹰流连，不如加快步伐、埋头向前

行动：每次迭代仅做**1次**元素比较，只有**2个**分支方向（放弃**命中**的分支）

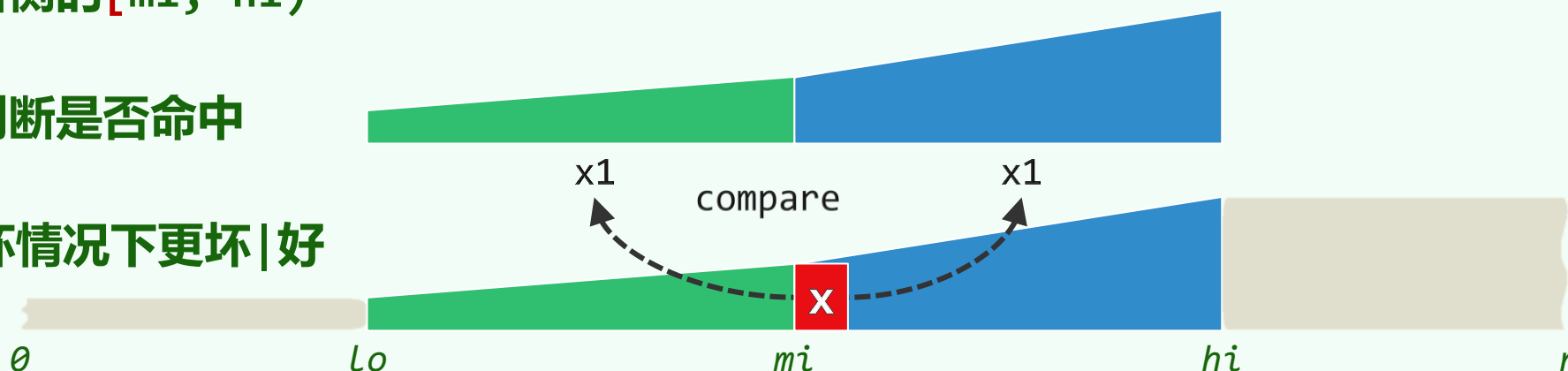
- $e < x$ ：则深入左侧的 $[lo, mi)$

- $x \leq e$ ：则深入右侧的 $[mi, hi)$

❖ 直到 $hi-lo = 1$ ，才判断是否命中

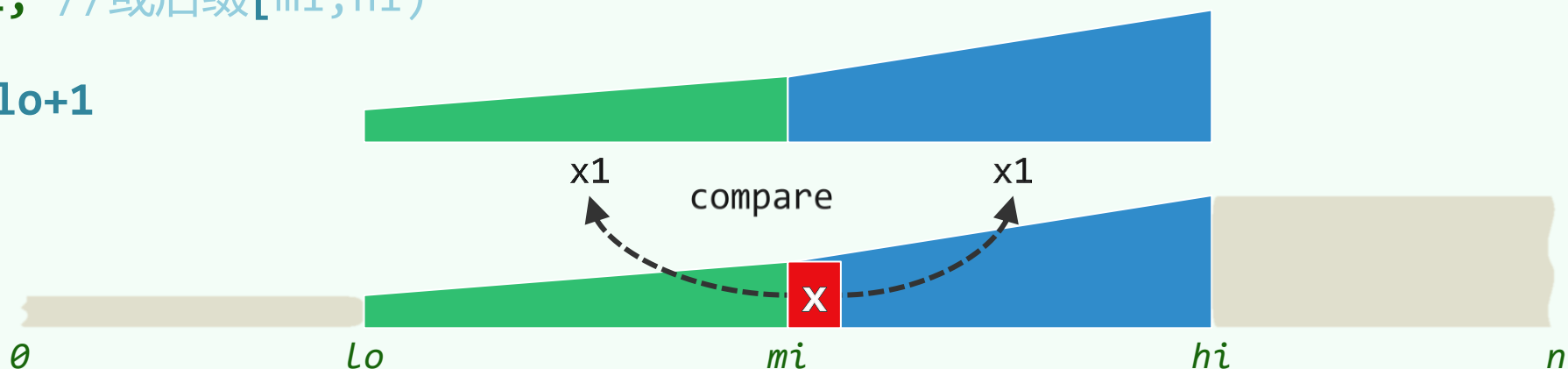
❖ 相对于版本A，最好|坏情况下更坏|好

整体性能有所提高



# 实现

```
template <typename T> Rank binSearch(T* S, const T& e, Rank lo, Rank hi) {
 if (e < S[lo]) return lo - 1; //相当于e = -inf
 while (1 < hi - lo) { //每步迭代一次比较、两个分支; 不能提前终止
 Rank mi = (lo + hi) / 2; //以中位数为轴点, 区间宽度折半
 if (e < S[mi]) hi = mi; //经比较后确定深入前缀[lo,mi)
 else lo = mi; //或后缀[mi,hi)
 } //出口时hi = lo+1
 return lo;
} //binSearch
```



# 返回值

❖ 验证: binSearch()的版本B, 总是返回

- $m = \max(\{-1\} \cup \{k \mid S[k] \leq e\})$
- 不大于e的最大者

❖ 得益于此, 有序向量才可以便捷地维护

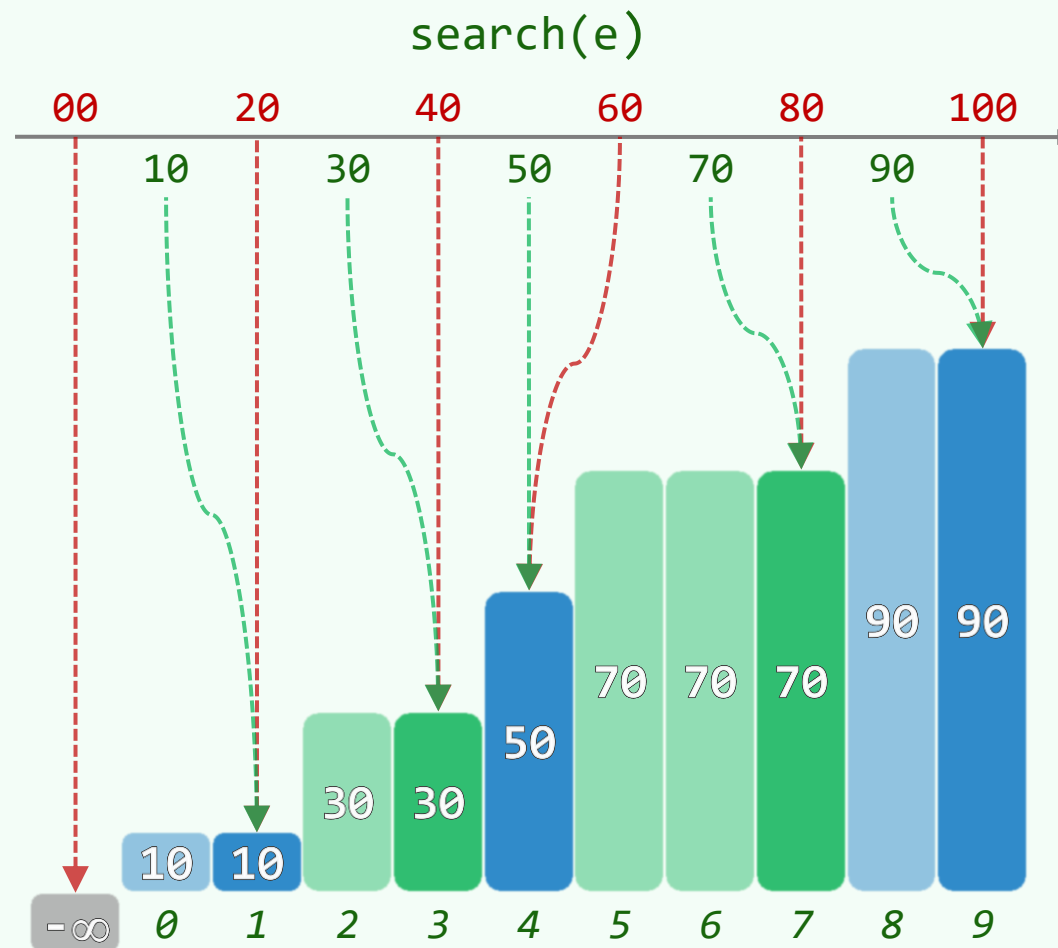
`V.insert( 1 + V.search(e), e )`

❖ 有序性: 即便失败, 也能给出e可安置的位置

稳定性: 相等的元素按插入次序排列

先到者居前, 晚来者靠后

❖ 然而, 算法仍欠紧凑, ASL还可优化...



向量

有序向量：二分查找（版本c）

02-C4

邓俊辉

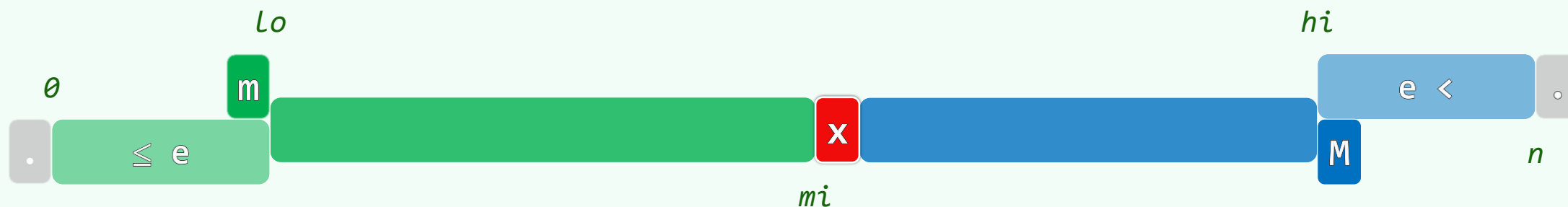
Outward failure may be a manifested variant of inward success.

deng@tsinghua.edu.cn

# 实现

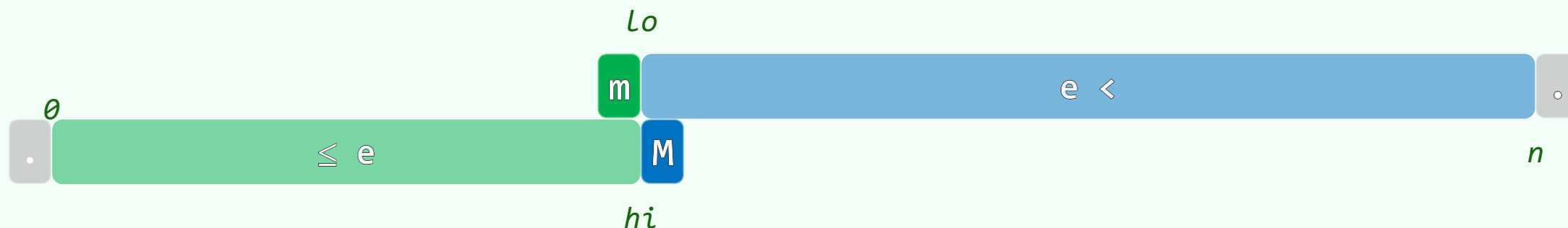
```
template <typename T> Rank binSearch(T* S, const T& e, Rank lo, Rank hi) {
 while (lo < hi) { //不变性: $[0, lo) \leq e < [hi, n)$
 Rank mi = (lo + hi) / 2;
 if (e < S[mi]) hi = mi; //[lo, mi)
 else lo = mi + 1; //(mi, hi)
 } //出口时, $[lo = hi] = M$
 return lo - 1; //至此, $[lo]$ 为大于e的最小者, 故 $[lo-1] = m$ 即为不大于e的最大者
}
```

Loop Invariant:  $[0, lo) \leq e < [hi, n)$



❖ 只要该不变性始终保持，当算法终止 ( $lo == hi$ ) 时

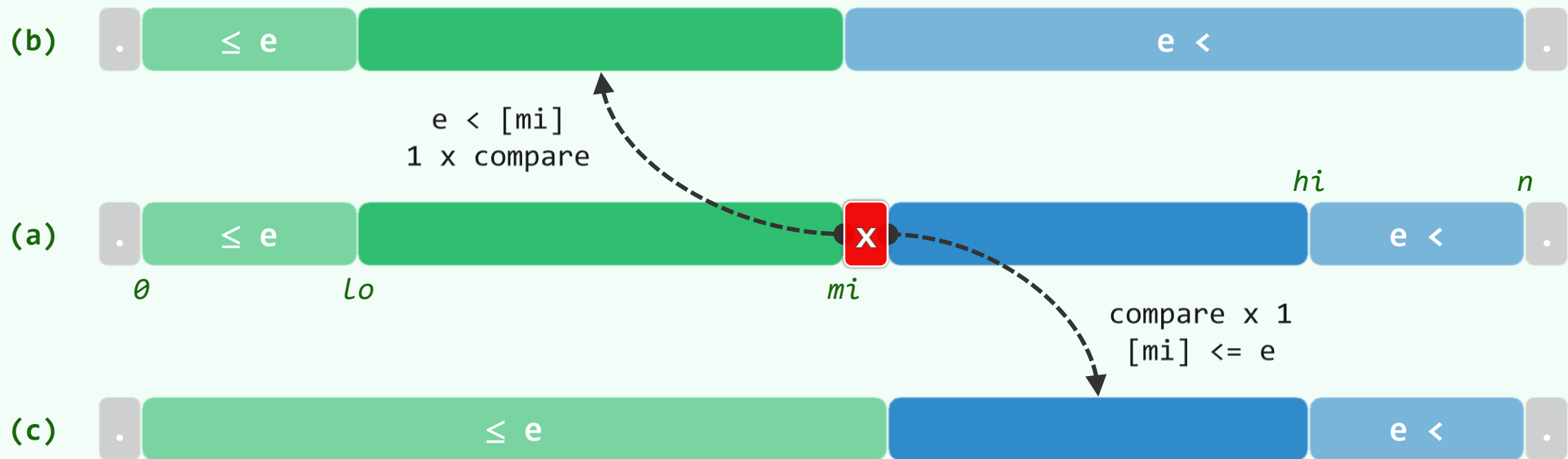
- $[lo-1]$  即是不大于  $e$  的最大者:  $m = \max(\{-1\} \cup \{k \mid S[k] \leq e\})$
- $[hi]$  即是不小于  $e$  的最小者:  $M = \min(\{k \mid e < S[k]\} \cup \{n\})$



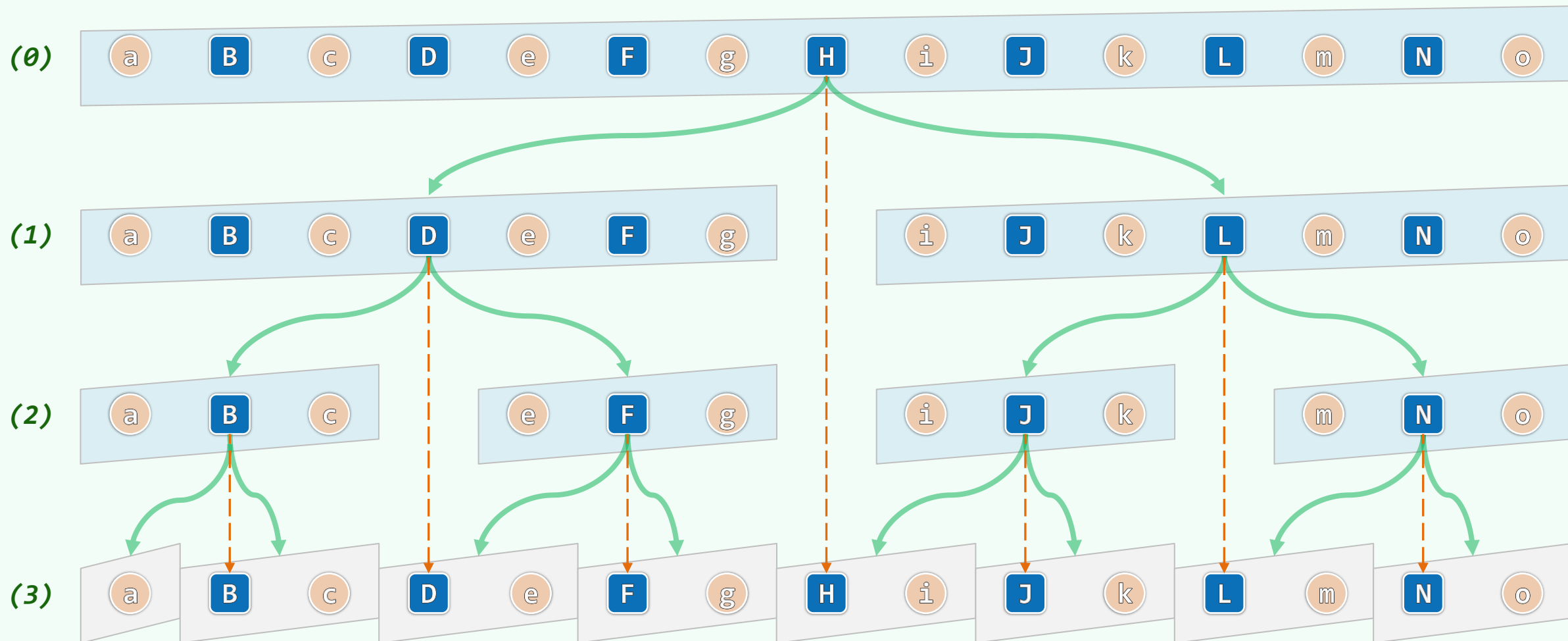
Loop Invariant:  $[0, lo) \leq e < [hi, n)$

❖ 初始时,  $lo = 0$ 且 $hi = n$ ,  $[0, lo) = [hi, n) = \emptyset$ , 自然成立

❖ 数学归纳: 假设不变性一直保持至(a); 则以下无非两种情况...



**平均查找长度:**  $ASL_{succ} = ASL_{fail} = \mathcal{O}(1.00 \cdot \log n)$





向量

有序向量：插值查找

02-C5

...那么在最后剩下的一万个猎手中，肯定有人会做出这样的选择：向那个位置开一枪试试...

你去用剃刀开采花岗岩吧，或用丝线系泊船，然后你兴许就能明白，我们无法指望用人类的知识和理性这样锋利却脆弱的工具，来对抗如巨人般的人类激情和傲慢

邓俊辉

deng@tsinghua.edu.cn

# 原理与算法

❖ 大数定律：越长的序列，元素的分布越有规律

❖ 最为常见：独立同分布且服从均匀分布

$[lo, hi]$ 内各元素的秩与其数值会近似地呈线性关系

$$\frac{mi - lo}{hi - lo} \approx \frac{[mi] - [lo]}{[hi] - [lo]}$$

❖ 因此：通过猜测轴点mi，可以极大地提高收敛速度

$$mi \approx lo + (hi - lo) \cdot \frac{e - [lo]}{[hi] - [lo]}$$

❖ 以英文词典为例：binary大致位于2/26处

search大致位于19/26处

|      |    |   |      |             |
|------|----|---|------|-------------|
| [lo] | 0  | A | 1    | [1,53)      |
|      | 1  | B | 74   | [53,104)    |
|      | 2  | C | 158  | [104,156)   |
|      | 3  | D | 292  | [156,208)   |
|      | 4  | E | 368  | [208,259)   |
|      | 5  | F | 409  | [259,311)   |
|      | 6  | G | 473  | [311,363)   |
|      | 7  | H | 516  | [363,414)   |
|      | 8  | I | 562  | [414,466)   |
|      | 9  | J | 607  | [466,518)   |
|      | 10 | K | 617  | [518,569)   |
|      | 11 | L | 628  | [569,621)   |
|      | 12 | M | 681  | [621,673)   |
|      | 13 | N | 748  | [673,724)   |
|      | 14 | O | 771  | [724,776)   |
|      | 15 | P | 806  | [776,827)   |
|      | 16 | Q | 915  | [827,879)   |
|      | 17 | R | 922  | [879,931)   |
|      | 18 | S | 1002 | [931,982)   |
|      | 19 | T | 1176 | [982,1034)  |
|      | 20 | U | 1253 | [1034,1086) |
|      | 21 | V | 1271 | [1086,1137) |
|      | 22 | W | 1289 | [1137,1189) |
|      | 23 | X | 1337 | [1189,1241) |
|      | 24 | Y | 1338 | [1241,1292) |
|      | 25 | Z | 1341 | [1292,1344) |
| [hi] | 26 |   | 1344 |             |

## 实例：查找目标 $e = 50$

❖  $lo = 0, hi = 18$

| 0 | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 |
|---|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 5 | 10 | 12 | 14 | 19 | 23 | 29 | 36 | 39 | 41 | 44 | 51 | 54 | 59 | 72 | 79 | 82 | 86 | 92 |

插值:  $mi = 0 + (18 - 0) * (50 - 5) / (92 - 5) = 9$

比较:  $A[9] = 41 < e$

❖  $lo = 10, hi = 18$

| 0 | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 |
|---|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 5 | 10 | 12 | 14 | 19 | 23 | 29 | 36 | 39 | 41 | 44 | 51 | 54 | 59 | 72 | 79 | 82 | 86 | 92 |

插值:  $mi = 10 + (18 - 10) * (50 - 44) / (92 - 44) = 11$

比较:  $A[11] = 51 > e$

❖  $lo = hi = 10$

| 0 | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 |
|---|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 5 | 10 | 12 | 14 | 19 | 23 | 29 | 36 | 39 | 41 | 44 | 51 | 54 | 59 | 72 | 79 | 82 | 86 | 92 |

插值:  $mi = 10$

比较:  $A[10] = 44 < e$ , 故返回: NOT\_FOUND

# 性能

❖ 最坏:  $hi - lo = \mathcal{O}(n)$

//具体实例?

❖ 平均: 每经一次比较, 待查找区间宽度由  $n$  缩至  $\sqrt{n}$  // [Yao76, PIA78], 习题解析[2-24]

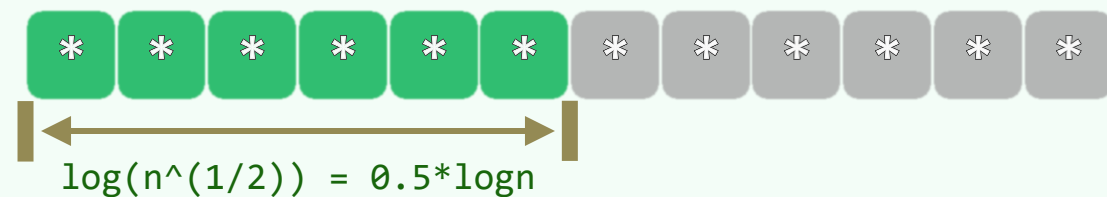
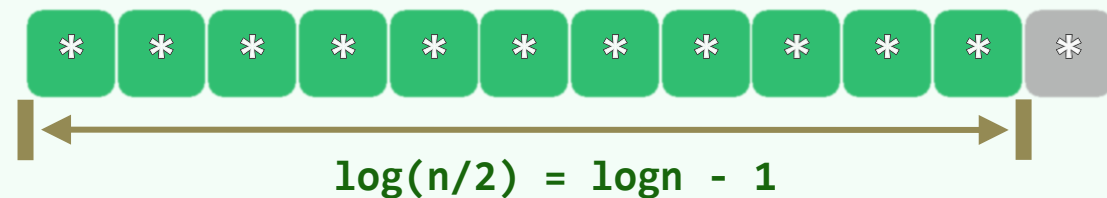
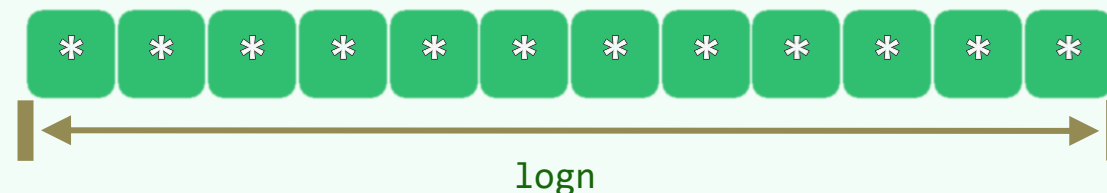
$$n \rightarrow \sqrt{n} \rightarrow \sqrt{\sqrt{n}} \rightarrow \sqrt{\sqrt{\sqrt{n}}} \rightarrow \dots \rightarrow 2$$

$$\underbrace{n \rightarrow n^{1/2^1} \rightarrow n^{1/2^2} \rightarrow n^{1/2^3} \rightarrow \dots \rightarrow 2}_{\mathcal{O}(\log \log n)}$$

❖ 每经一次比较,

查找区间宽度的数值  $n$  开方, 有效字长  $\log n$  减半

- 插值查找 = 在字长意义上的折半查找
- 二分查找 = 在字长意义上的顺序查找



# 综合评价

❖ 从 $O(\log n)$ 到 $O(\log \log n)$ , 优势**并不**明显

(除非查找表极长, 或比较操作成本极高)

比如,  $n = 2^{(2^5)} = 2^{32} = 4G$  时

- $\log_2(n) = 32$

- $\log_2(\log_2(n)) = 5$

❖ 须引入**乘法**、**除法**运算

❖ 易受**畸形分布**的干扰和“蒙骗”

❖ 实际可行的方法是

算法**接力**

- 首先通过**插值**查找

迅速将查找范围缩小到一定的尺度

- 然后再改为**二分**查找

进一步缩小范围

- 最后 (当数据项只有200 ~ 300时)

改用**顺序**查找

向量

起泡排序

邓俊辉

deng@tsinghua.edu.cn

通过感觉，物体会——呈现在我们的面前，就像在自然中一样；  
通过比较，我重新安排或者调整它们的顺序

十五年中，这古园的形体被不能理解它的人肆意雕琢，幸好有些  
东西是任谁也不能改变它的

吾日三省吾身：為人謀而不忠乎？與朋友交而不信乎？傳不習乎？

# 排序器：统一入口

```
template <typename T> void Vector<T>::sort(Rank lo, Rank hi) {
 if (lo + 2 > hi) return; //退化情况，统一处理
 switch (rand() % 6) {
 case 1 : bubbleSort(lo, hi); break; //起泡排序
 case 2 : selectionSort(lo, hi); break; //选择排序 (习题)
 case 3 : mergeSort(lo, hi); break; //归并排序
 case 4 : heapSort(lo, hi); break; //堆排序 (第12章)
 case 5 : quickSort(lo, hi); break; //快速排序 (第14章)
 default : shellSort(lo, hi); break; //希尔排序 (第14章)
 } //随机选择算法，以尽可能充分地测试；应用时可视具体问题的特点，灵活确定或扩充
} //sort
```

## 基本版：用模板封装一下

❖ `template <typename T>`

```
void Vector<T>::bubbleSort(Rank lo, Rank hi) {
```

```
 while (lo < --hi) //逐趟起泡扫描
```

```
 for (Rank i = lo; i < hi; i++) //逐对检查相邻元素
```

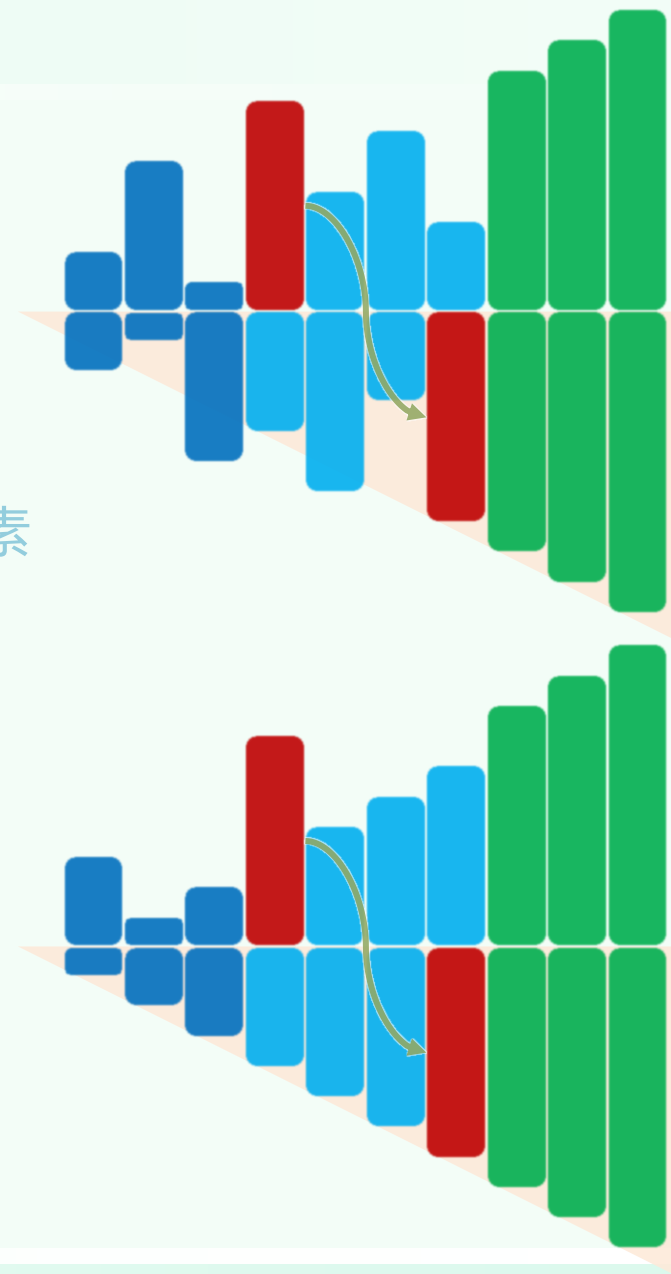
```
 if (_elem[i] > _elem[i + 1]) //若逆序
```

```
 swap(_elem[i], _elem[i + 1]); //则交换
```

```
}
```

❖  $\Theta(n^2)$ ：不识好坏，千篇一律

❖ 然而，不同输入的**混乱**程度毕竟有别，比如...





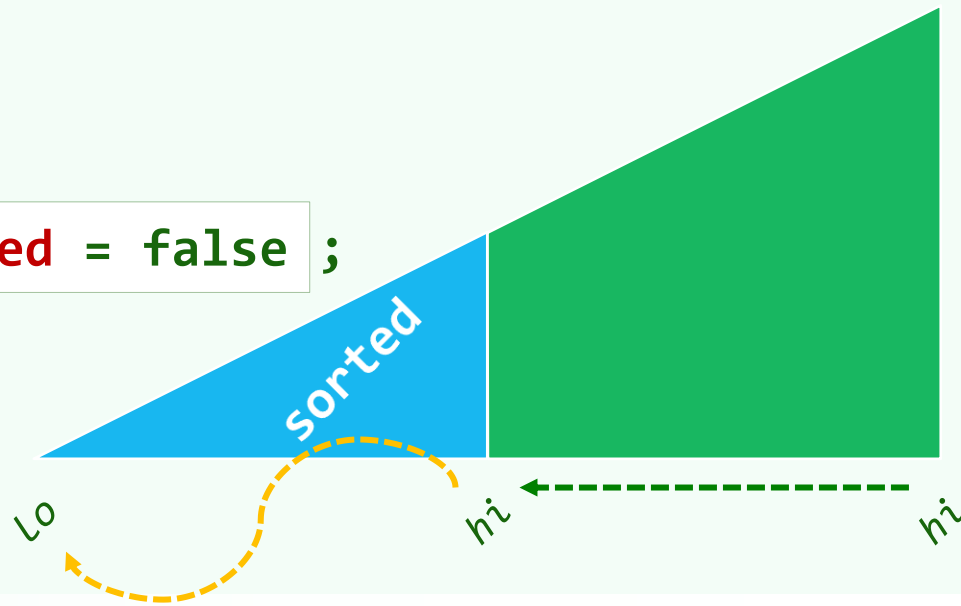
# 提前终止版

❖ 有可能...行至中途, 前缀已然提前有序| **sorted**——此时, 可以提前收工!

❖ 关键: 如何及时发现这类情况, 同时渐近复杂度总体不增呢?

```
❖ template <typename T> void Vector<T>::bubbleSort(Rank lo, Rank hi) {
 for (bool sorted = false ; sorted = !sorted ; hi--)
 for (Rank i = lo + 1; i < hi; i++)
 if (_elem[i-1] > _elem[i])
 swap(_elem[i-1], _elem[i]), sorted = false ;
}
```

❖ 有改进, 但仍有继续改进的余地, 比如...



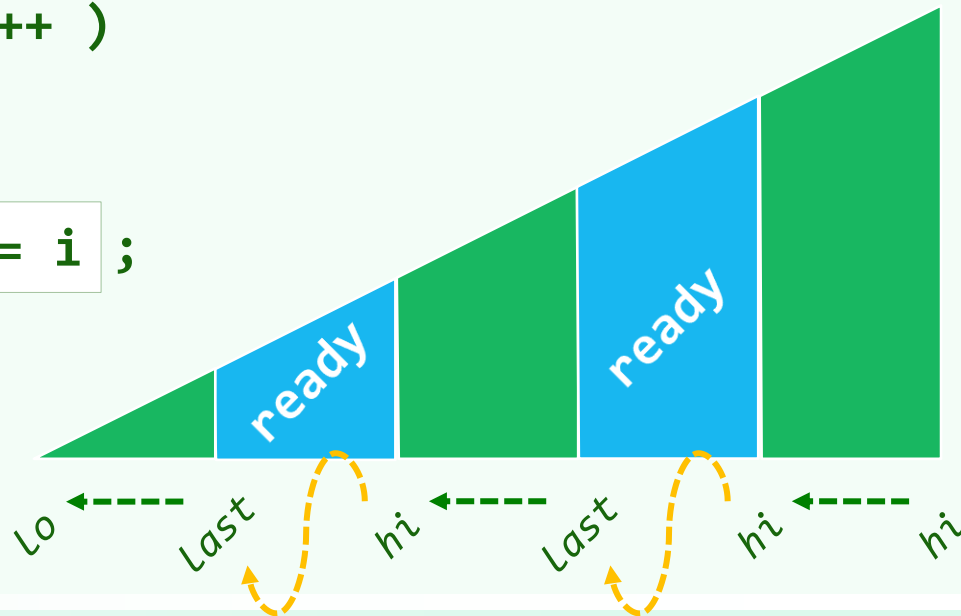
# 跳跃版

❖ 在上一趟扫描过程中，若**最后**被交换的是 $[last-1]$ 和 $[last]$ ，则 $[last, hi)$ 必然均已就位

❖ 此时，可以**跳过**这个后缀，直接对 $[lo, last)$ 继续做起泡扫描

```
❖ template <typename T> void Vector<T>::bubbleSort(Rank lo, Rank hi) {
 for (Rank last ; lo < hi; hi = last)
 for (Rank i = (last = lo) + 1; i < hi; i++)
 if (_elem[i-1] > _elem[i])
 swap(_elem[i-1], _elem[i]), last = i ;
}
```

❖ 断言:  $[lo, last) < [last] \leq (last, hi)$



# 综合评价

❖ 时间效率：最好 $O(n)$ ，最坏 $O(n^2)$

❖ 排序算法的**稳定性**|stability，是更为细致的要求

**相等**的元素在输入、输出序列中的**相对次序**，是否保持**不变**？

- 输入： 6, 7a, 3, 2, 7b, 1, 5, 8, 7c, 4

- 输出： 1, 2, 3, 4, 5, 6, 7a, 7b, 7c, 8 //stable

1, 2, 3, 4, 5, 6, 7a, 7c, 7b, 8 //unstable

❖ 以上起泡排序算法是**稳定**的吗？是的！

- 排序过程中，唯有**相邻**元素才可交换

- 比较相邻元素时，是用严格不等号" $>$ "，而非" $>=$ "

02-E

向量

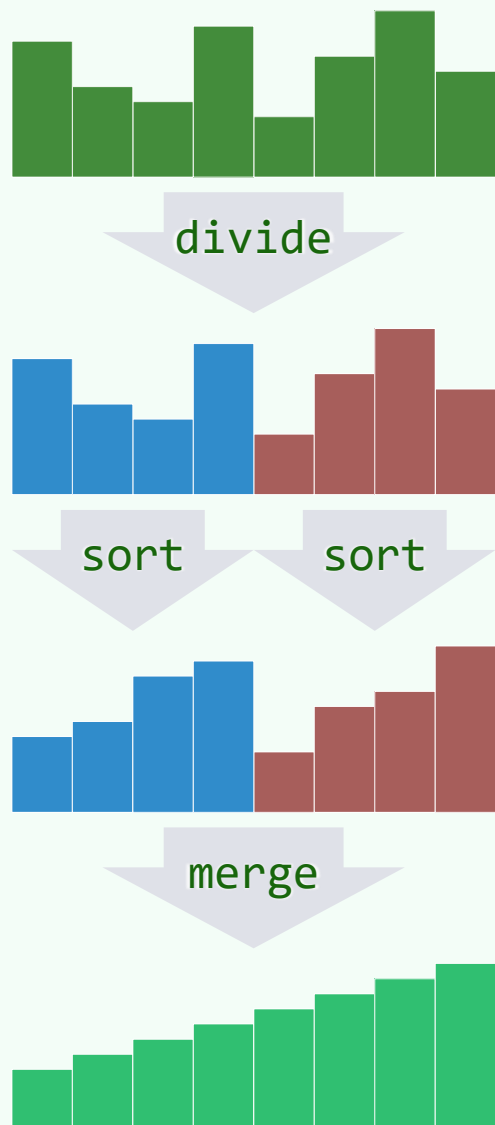
归并排序

邓俊辉

[deng@tsinghua.edu.cn](mailto:deng@tsinghua.edu.cn)

天下大势，分久必合，合久必分

# 分而治之：原理



❖ 向量与列表通用

❖ J. von Neumann

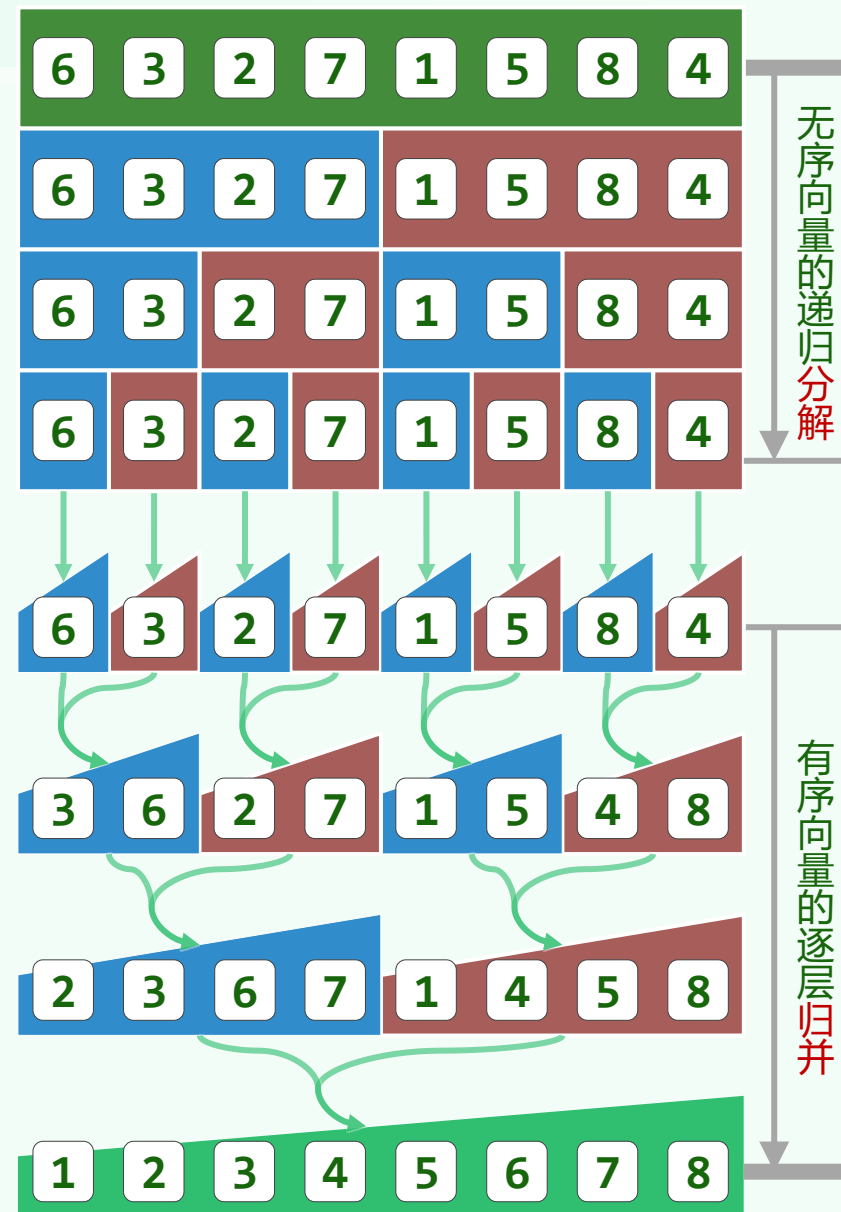
1945年首次编程实现



- 序列均分为二  $// O(1)$
- 子序列递归排序  $// 2 \times T(n/2)$
- 合并有序子序列  $// O(n)$

❖ 前两步毫无技巧可言

最后一步才是实质的



# 分而治之：主算法

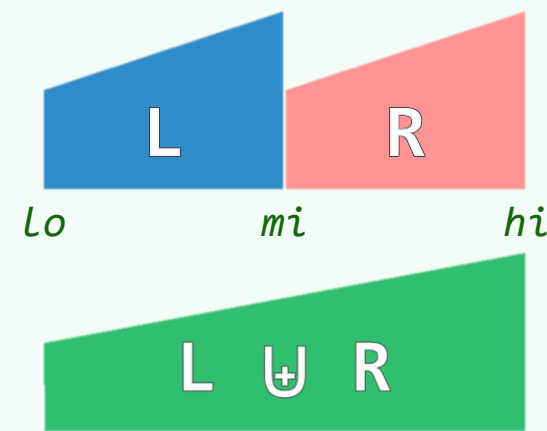
```
❖ template <typename T> void Vector<T>::mergeSort(Rank lo, Rank hi) {
 if (hi - lo < 2) return; //单元素自然有序, 否则

 Rank mi = (lo + hi) / 2; //均分为前缀、后缀

 mergeSort(lo, mi); mergeSort(mi, hi); //各自排序

 merge(lo, mi, hi); //归并 (实质的计算) , O(n)

} //O(nlogn)
```

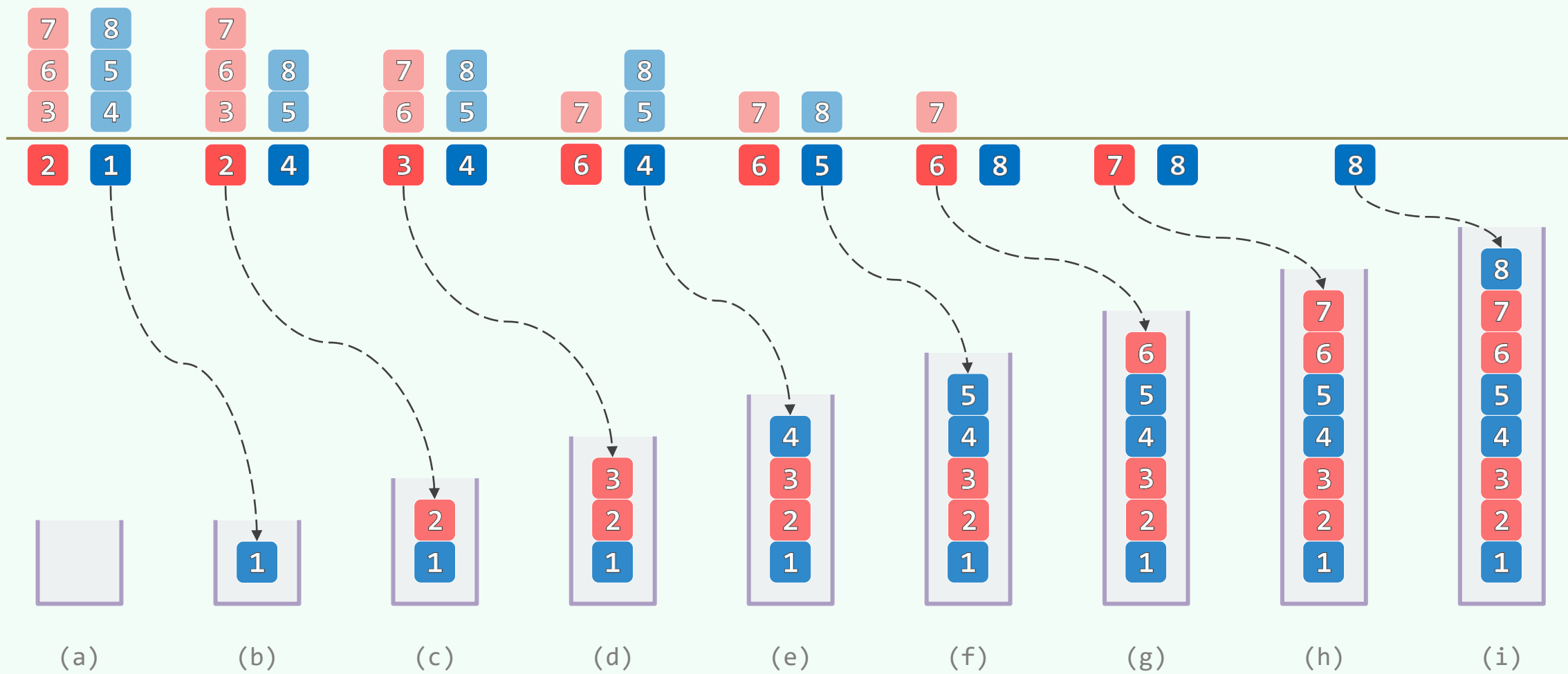


❖ 我们马上就会看到，**归并**能在**线性**时间内完成 //不仅**向量**，**列表**也可以！

那么整体复杂度即为  $\mathcal{O}(n \cdot \log n)$  ! //worst-case optimal

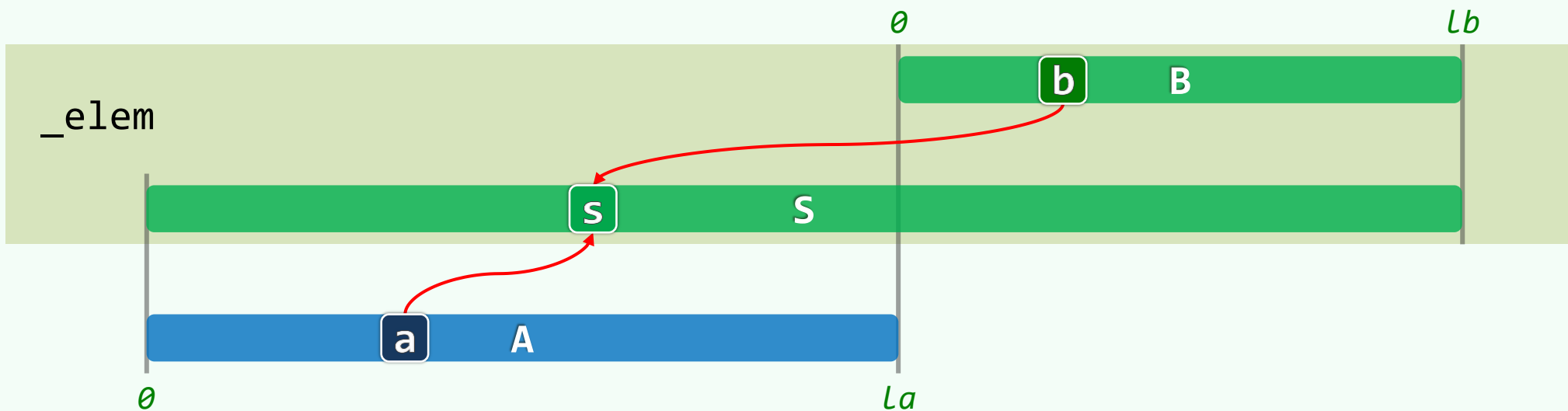
❖ 依然是...**减治**：首先，关注全局最小的那个元素...

## 二路归并：减而治之



## 二路归并：实现 (1/2)

```
template <typename T> void Vector<T>::merge(Rank lo, Rank mi, Rank hi) {
 Rank s = 0; T* S = _elem + lo; //就地归并至序列S[0,hi-lo)
 Rank a = 0, la = mi - lo; T* A = new T[la]; //A[0,la)复制自
 for (Rank i = 0; i < lb; i++) A[i] = S[i]; //s的前缀
 Rank b = 0, lb = hi - mi; T* B = S + la; //B[0,lb)就是S的后缀
```





## 二路归并：实现

```
while ((a < la) && (b < lb)) //反复地比较A、B的首元素
```

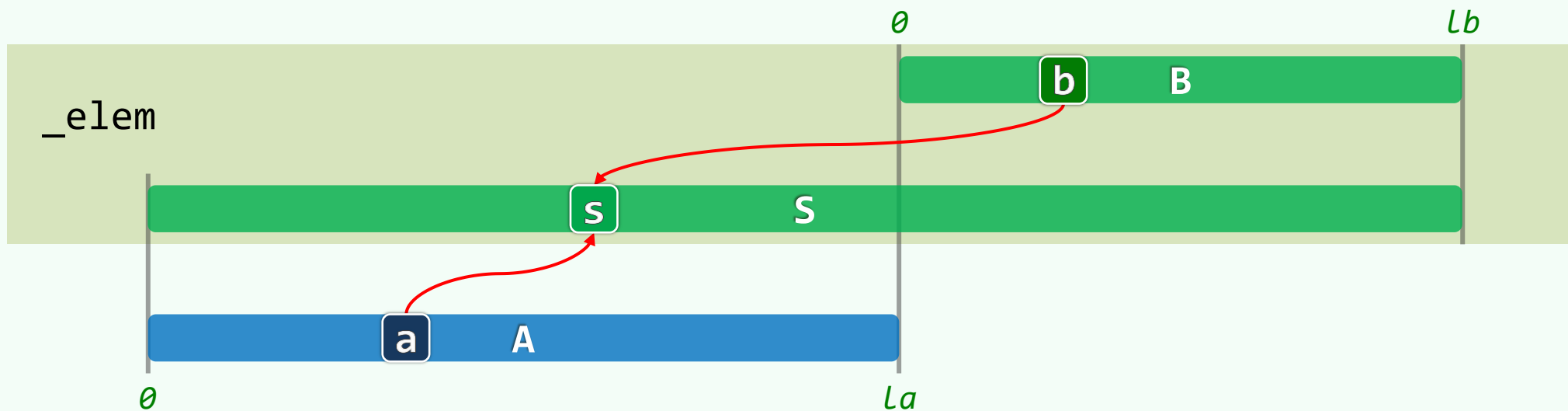
```
 S[s++] = (A[a] <= B[b]) ? A[a++] : B[b++]; //小者优先归入S
```

```
while (a < la) //若B先耗尽，则
```

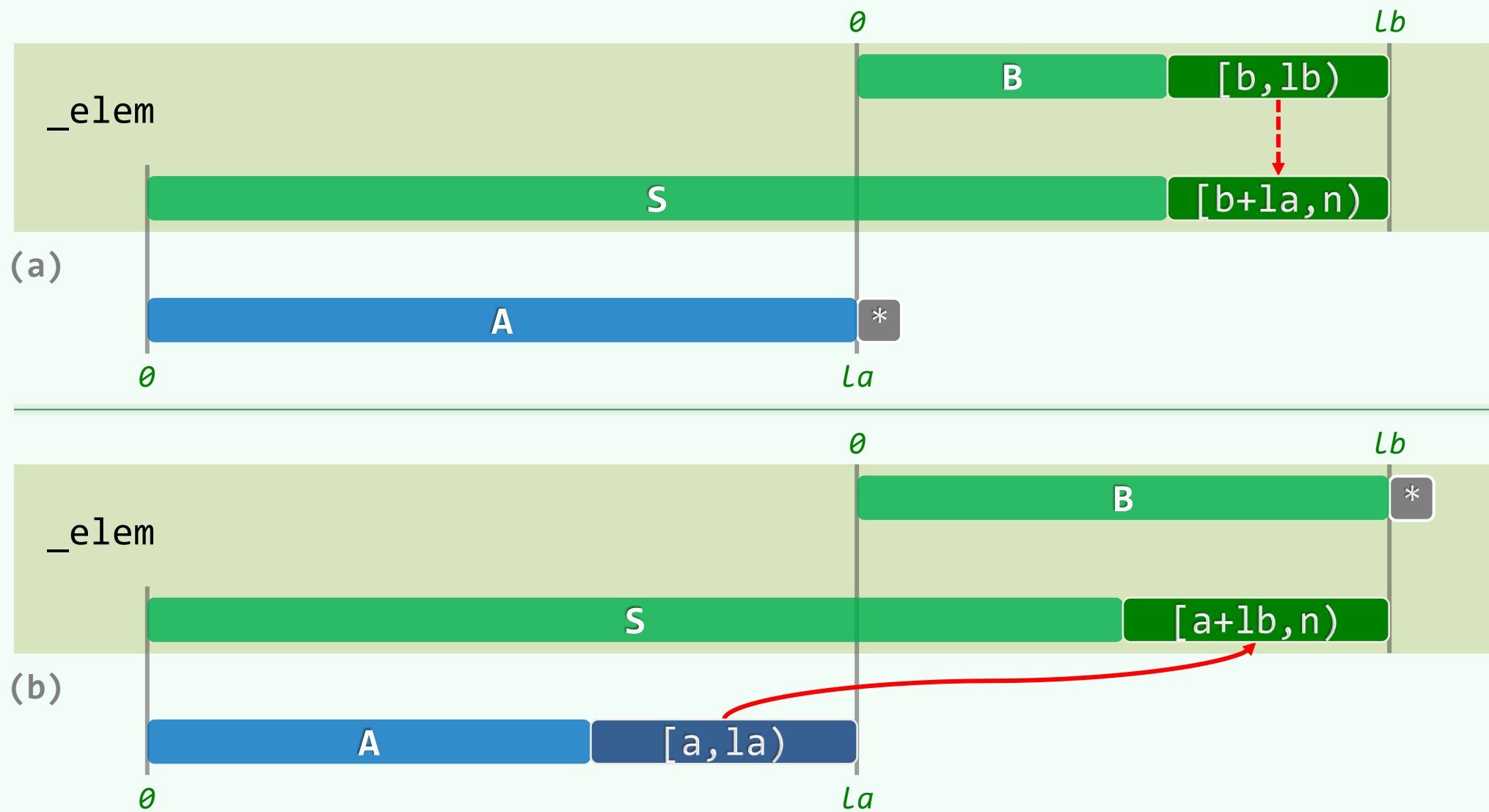
```
 S[s++] = A[a++]; //将A残余的后缀归入S中——若A先耗尽呢？
```

```
delete[] A; //new和delete很耗时，如何减少？
```

```
}
```



## 二路归并：正确性



# 运行时间

❖ 二路归并中，两个while循环每迭代一步  
i都会递增；j或k中**之一**也会随之递增

❖ 因：  $0 \leq j \leq lb$ ,  $0 \leq k \leq lc$

故： 累计迭代步数  $\leq lb + lc = n$

```
while ((j < lb) && (k < lc))
 A[i++] = (B[j] <= C[k]) ? B[j++] : C[k++];

while (j < lb)
 A[i++] = B[j++];
```

merge()只需  $\mathcal{O}(n)$  时间——即便lb和lc**不相等**，甚至相差**悬殊**，这一结论依然成立

❖ 由主定理，mergeSort()的时间复杂度为  $\mathcal{O}(n \log n)$

❖ 这一算法的思路及结论，也适用于另一类序列——列表（下一章）

## 优点 + 不足

❖ 最坏情况最优  $\mathcal{O}(n \log n)$  的首个排序算法

❖ 不需随机读写，完全顺序访问——尤其  
适用于列表之类的序列、磁带之类的设备

❖ 只要实现恰当，可保证稳定

❖ 可扩展性极佳，十分适宜于外部排序

❖ 同层子任务彼此独立，易于并行化

❖ 反复new/delete，耗时

——可优化为只做一次

❖ 需要  $\Omega(n)$  辅助空间

——适当多花时间，可令就地

❖ 即便已接近甚至完全有序，仍需  $\Omega(n \log n)$  时间

——可优化至  $\mathcal{O}(n)$ ，同时其它情况不受影响

❖ 序列足够短之后，递归效率骤降

——适时变招，改用`bubbleSort()`

向量

位图：结构

$\theta_2 - F_1$

这样做能保存的信息量就小多了，不到原来的  
万分之一，但他们也只能接受这个结果

邓俊辉

deng@tsinghua.edu.cn

# 有限整数集：秩 ~ 整数

$\forall 0 \leq k < U :$

$k \in S ?$       **bool**   test( Rank  $k$  );

$S \cup \{k\}$       **void**    set( Rank  $k$  );

$S \setminus \{k\}$       **void** clear( Rank  $k$  );



# 结构

```
class Bitmap {
```

```
 private:
```

```
 unsigned char * M;
```

```
 Rank N, _sz;
```

```
 public:
```

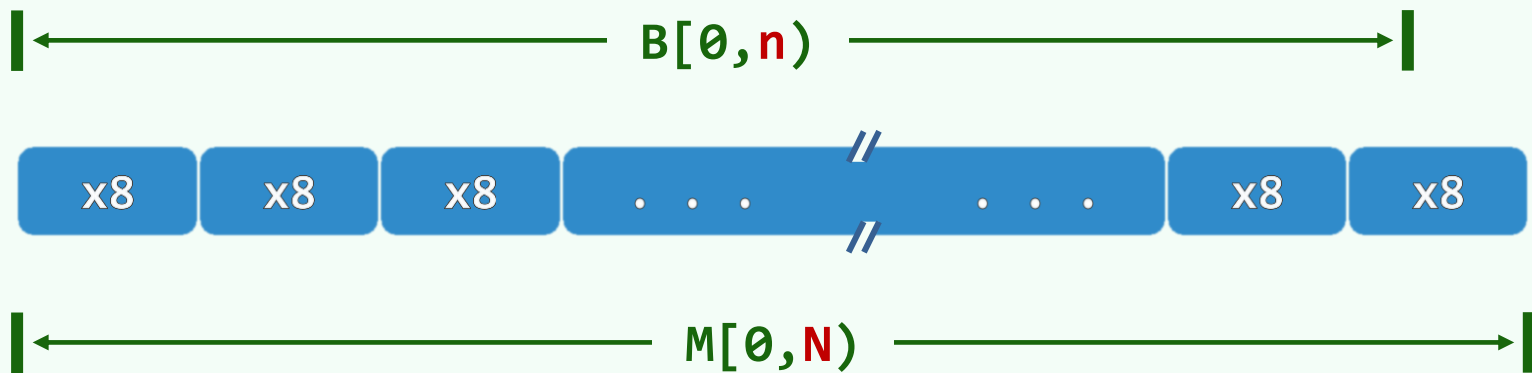
```
 Bitmap(Rank n = 8)
```

```
 { M = new uint8_t[N = (n+7)/8]; memset(M, 0, N); _sz = 0; }
```

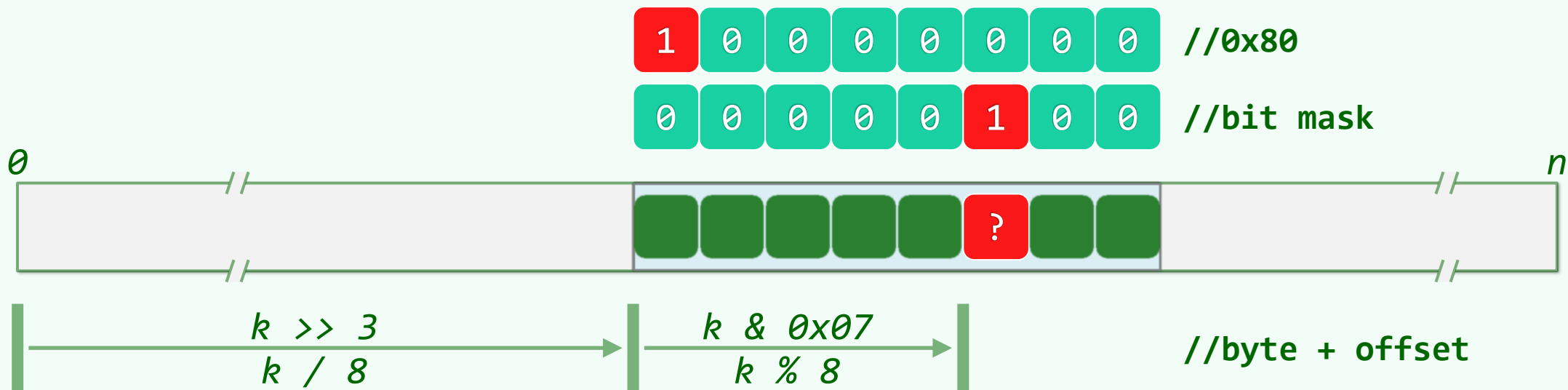
```
 ~Bitmap() { delete[] M; M = NULL; _sz = 0; }
```

```
 void set(Rank k); void clear(Rank k); bool test(Rank k);
```

```
};
```



# 实现



```
bool test (Rank k) { expand(k); return M[k>>3] & (0x80 >> (k&0x07)); }
void set (Rank k) { if (test(k)) return; M[k>>3] |= (0x80 >> (k&0x07)); _sz++; }
void clear(Rank k) { if (!test(k)) return; M[k>>3] &= ~(0x80 >> (k&0x07)); _sz--; }
```



向量

位图：典型应用

$\theta_2 - F_2$

邓俊辉

deng@tsinghua.edu.cn

Those too big to pass through are our friends.

# 去重：大数据+小集合

❖ 老问题：int A[n]的元素均取自[0, m)

如何剔除其中的相等者？

❖ 仿照Vector::uniquify()

先排序，再扫描

——  $\mathcal{O}(n \log n + n)$  —— 毫无压力

❖ 新特点：数据体量虽大，却有大量元素相等

- 想想我们电脑里的jpg、mp3、mp4...
- 还有，朋友圈...

❖ 比如，10,000,000,000个24位无符号整数

$$2^{24} = m \ll n = 10^{10}$$

必有大量的相等者

❖ 如果采用内部排序算法

至少需要  $4 * n = 40GB$  内存

- 即便能够申请到这么多空间
- 频繁的I/O也将导致整体效率的低下

❖ 那么， $m \ll n$  的条件，又应如何加以利用？

## 去重：借助位图

```
Bitmap B(m); //O(m)
```

```
for (Rank i = 0; i < n; i++)
```

```
 B.set(A[i]); //O(n)
```

```
for (Rank k = 0; k < m; k++)
```

```
 if (B.test(k))
```

```
 output(k); //O(m)
```

❖ 总体运行时间 =  $\mathcal{O}(n + m) = \mathcal{O}(n)$

❖ 空间 =  $\mathcal{O}(m)$

- 就上例而言，降至：

$$2^{24} = 16 * 10^6 \text{ bit} \sim 2\text{MB} \ll 40\text{GB}$$

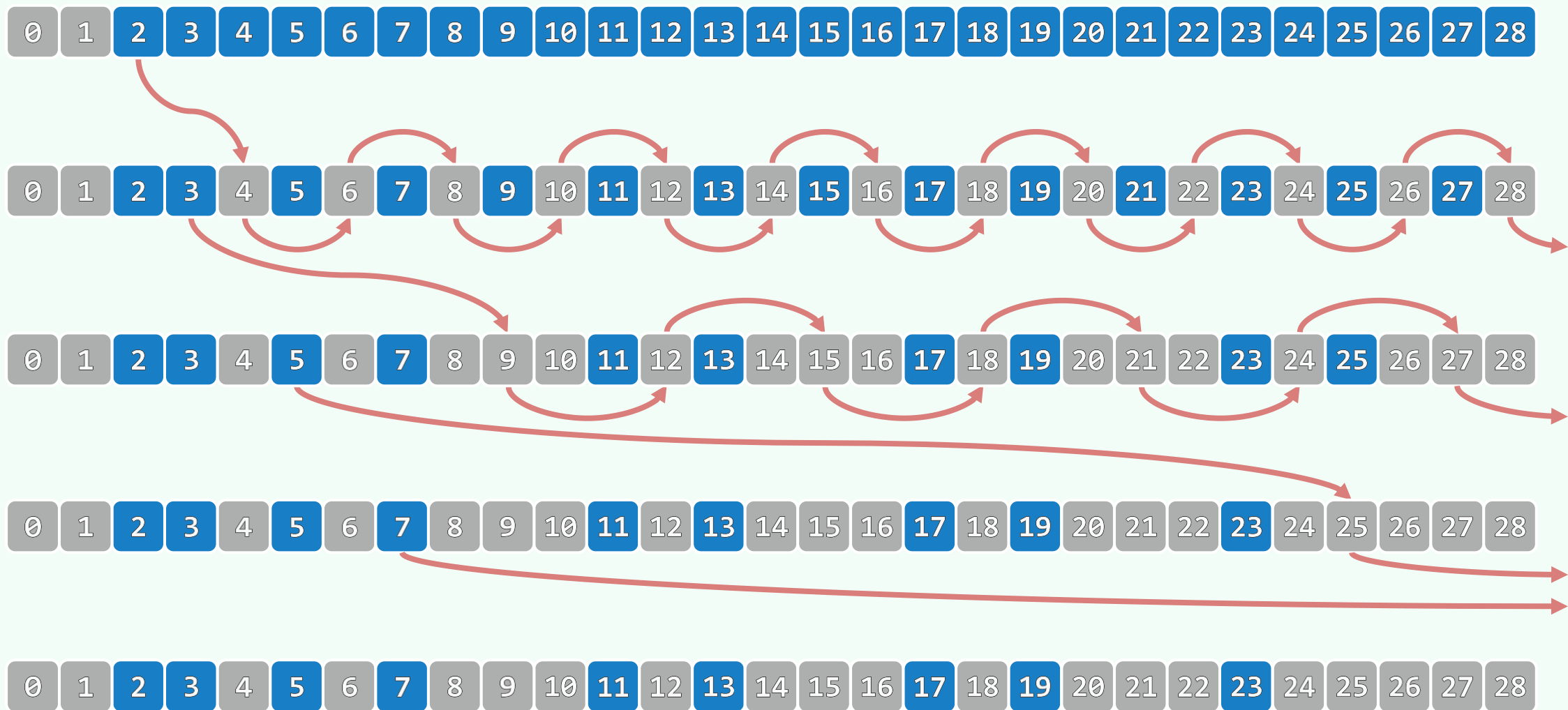
- 即便  $m = 2^{32}$ ，也不过：

$$2^{29} \text{ byte} = 0.5\text{GB}$$

❖ 关键在于，如何将查询词表

转换为某一整数集合 //保持兴趣

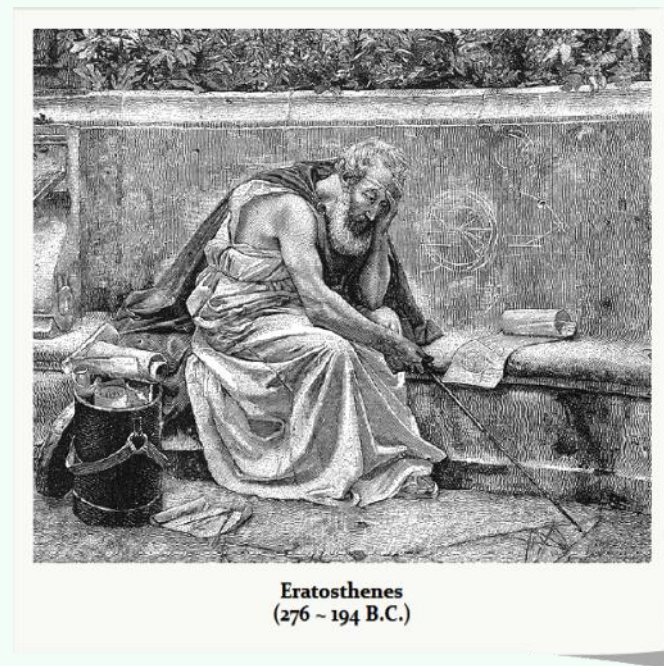
## 筛法：思路



# 筛法：实现

```
❖ void Eratosthenes(Rank n, char * file) {
 Bitmap B(n, true); B.clear(0); B.clear(1);
 for (Rank i = 2; i < n; i++)
 if (B.test(i))
 for (Rank j = i*i; j < n; j += i)
 B.clear(j);

 B.dump(file);
}
```

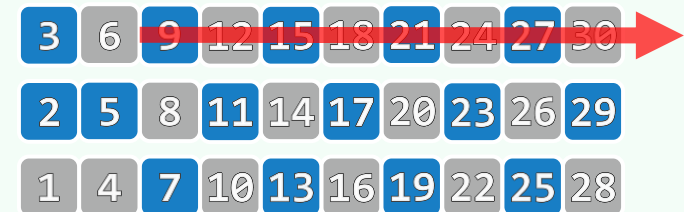
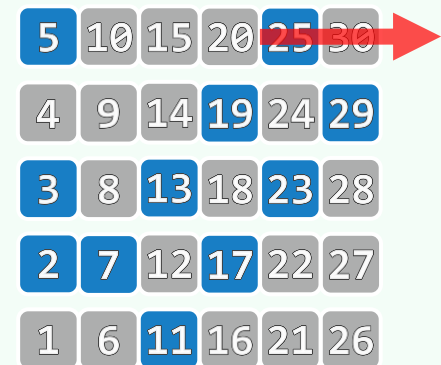


# 筛法：复杂度

❖ 由**素数定理**，内循环启动  $n/\ln n$  次；每次迭代  $\mathcal{O}(n/i)$  步

❖ 记小于  $n$  的最大素数为  $p(n)$

$$\begin{aligned} & n/2 + n/3 + n/5 + n/7 + n/11 + \dots + n/p(n) \\ < n/2 + n/3 + n/4 + n/5 + n/6 + \dots + n/(n/\ln n) \\ &= \mathcal{O}(n \cdot (\ln(n/\ln n) - 1)) \\ &= \mathcal{O}(n \cdot \ln n - n \cdot \ln(\ln(n))) \\ &= \mathcal{O}(n \cdot \log n) \end{aligned}$$



向量

位图：快速初始化

我从那无比圣洁的河水那里  
走了回来，仿佛再生了一般  
正如新的树用新的枝叶更新  
一身洁净，准备就绪，就飞往星辰

是知契者書其信誓之言，而盟之濫觴，君臣之大約也。亦謂刻木剖而分之，  
君持其左，臣執其右，即昔之銅虎、竹使，今之銅魚，並契之遺象也

邓俊辉

deng@tsinghua.edu.cn

$$O(n) \sim O(1)$$

❖ Bitmap的构造函数中，**总**要对数据区**统一清零**

尽管是通过`memset(M, 0, N)`来完成，但与逐位清零一样，也需要  $O(N) = O(n)$  时间！

❖ 虽然这并不会影响以上实例的渐近复杂度，但并非所有场合都是如此，比如

- 第09章的散列表，初始化的复杂度将直接影响到整体效率
- 第13章的bc[]表构造算法，需要  $O(|\Sigma| + m) = O(s + m)$  时间

若能省去初始化，则只需  $O(m)$  时间

- 借助Bitmap，固然可以判断1,000,000个32位整数中是否存在**相等者**

但 $2^{32} = 4 \times 10^9$ 个bit位清零所需的时间，远远超过接下来对 $10^6$ 个整数的遍历

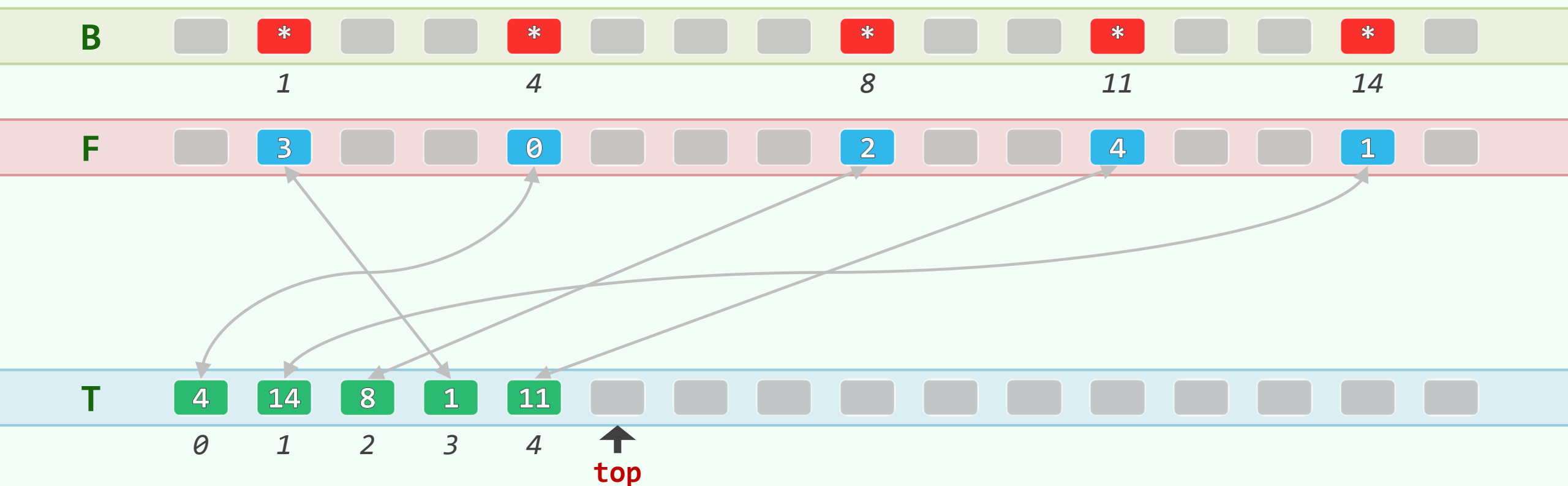
❖ 在这类场合，可否...**加速**...甚至...**免除**...数据区的初始化？



J. Hopcroft, 1974: Rank  $F[m]$ ,  $T[m]$ ,  $top = 0$ ;

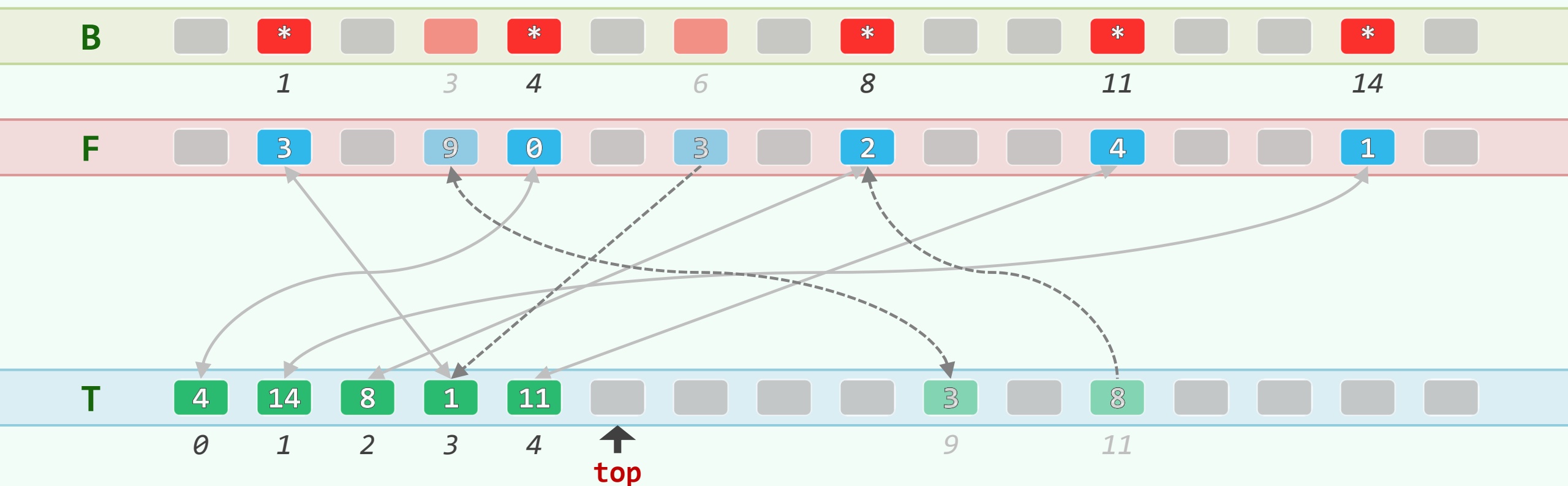
❖ 为区分三种状态，需要记录更多信息：拆分数组 + 校验环：

$To[From[k]] == k$  &  $From[To[i]] == i$  //君右杜左，虎符会合



测试:  $\text{test}(1|4|8|11|14) = \text{true}$  &  $\text{test}(3|6) = \text{false}$

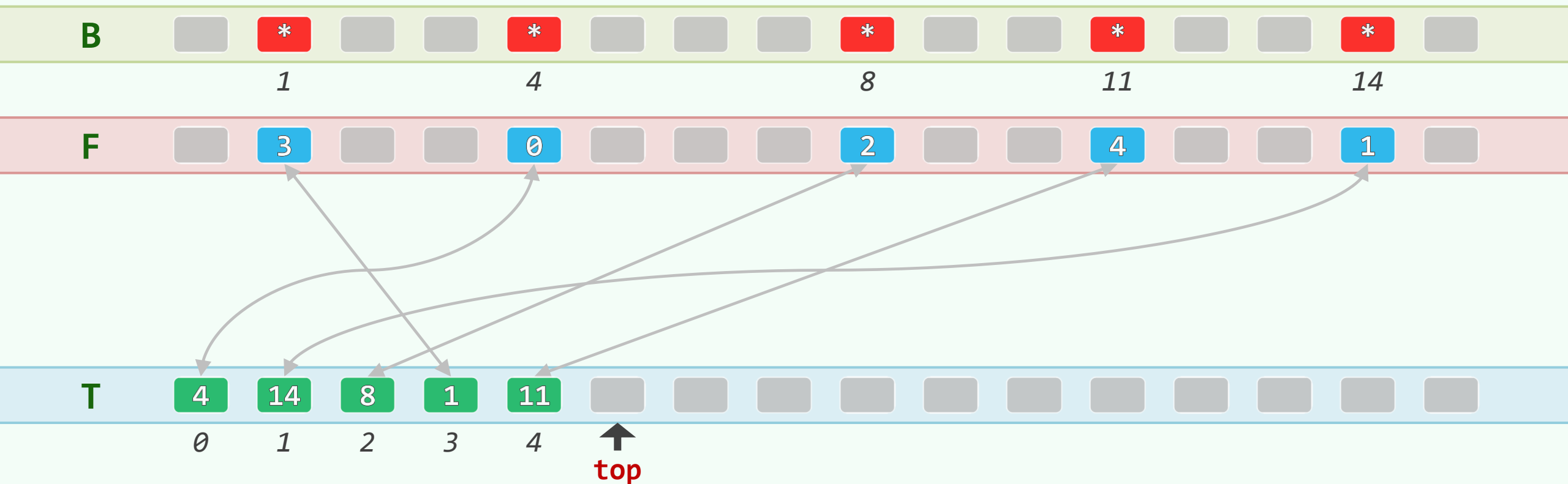
```
bool Bitmap::test(Rank k) //为剔除随机构成的校验环, 还需限制为前缀T[0,top)
{ return (F[k] < top) && (k == T[F[k]]); } //充要条件
```



## $O(1)$ 复位

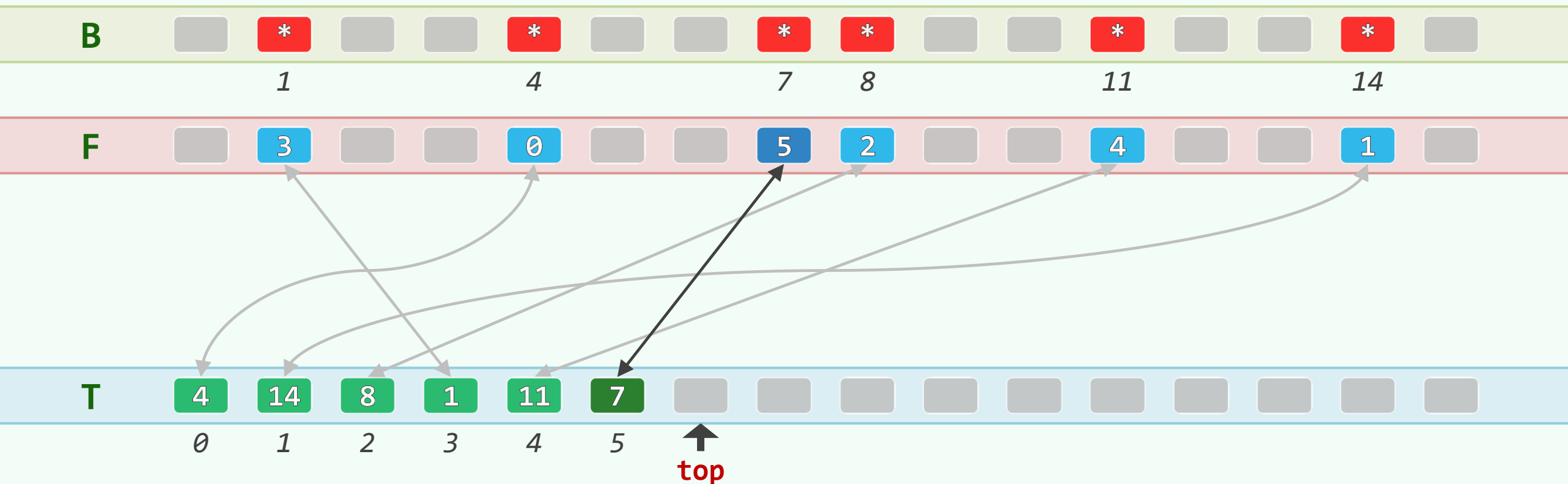
```
void Bitmap::reset() //前缀清空, 即是位图复位
```

```
{ top = 0; }
```



## $O(1)$ 插入: `set(7)`

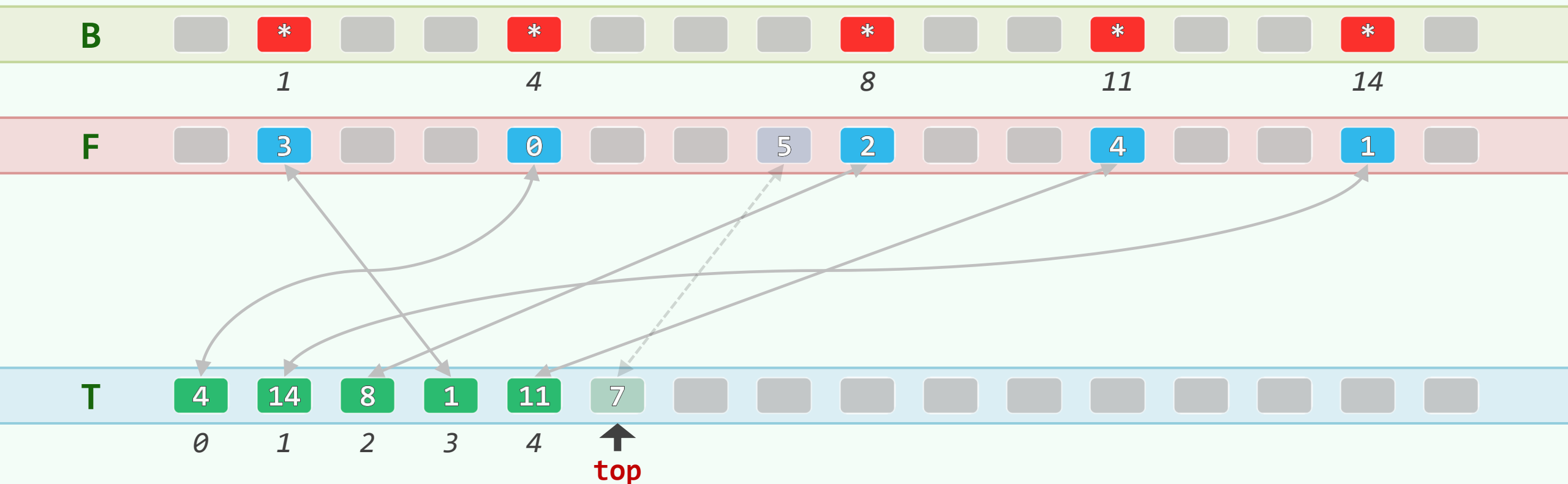
```
void Bitmap::set(Rank k) //在T尾部, 创建一个校验环
{ if (!test(k)) { T[top] = k; F[k] = top++; } }
```



$O(1)$ 删除：简单情况，删除最新创建的（在T中拖尾的）校验环：clear(7)

```
void Bitmap::clear(Rank k)
```

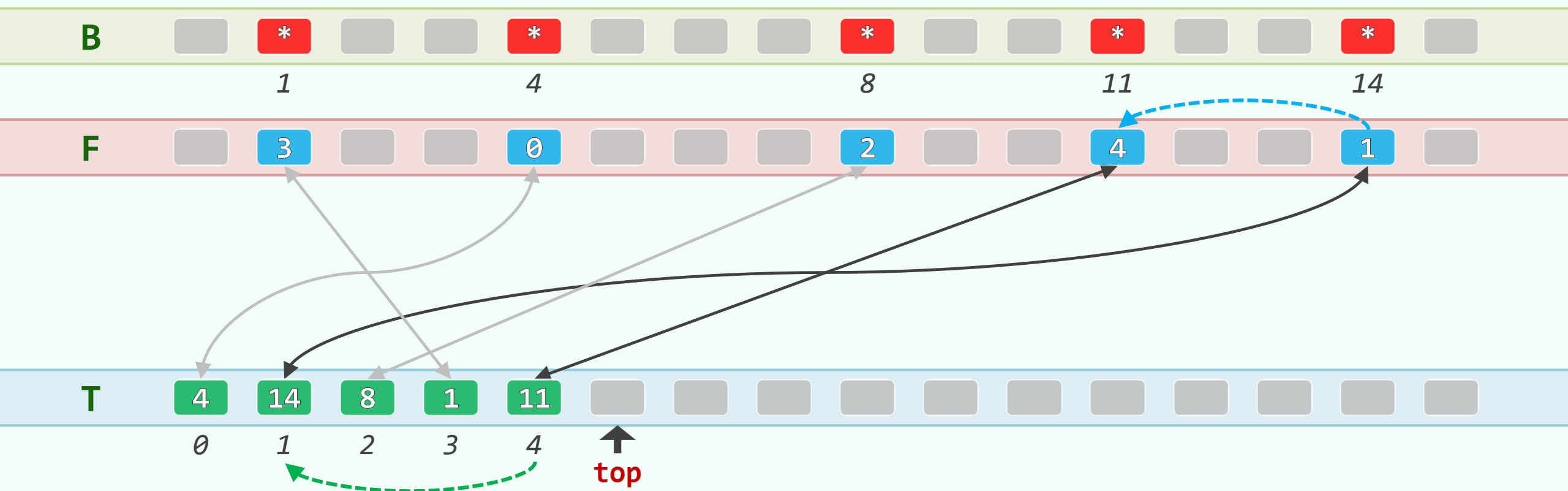
```
{ if (test(k) && (--top)) { F[T[top]] = F[k]; T[F[k]] = T[top]; } }
```



$O(1)$ 删除：一般情况，令校验环 $k$ ，与拖尾的校验环交换：clear(14) ...

```
void Bitmap::clear(Rank k)
```

```
{ if (test(k) && (--top)) { F[T[top]] = F[k]; T[F[k]] = T[top]; } }
```



$O(1)$ 删除: ... clear(14)

```
void Bitmap::clear(Rank k)
```

```
{ if (test(k) && (--top)) { F[T[top]] = F[k]; T[F[k]] = T[top]; } }
```

